```python
from google.colab import drive
drive.mount('/content/drive')

# Create working directory
import os
WORKING_DIR = '/content/drive/MyDrive/GenAI_Lab_Week10'
os.makedirs(WORKING_DIR, exist_ok=True)
os.makedirs(f'{WORKING_DIR}/models', exist_ok=True)
os.makedirs(f'{WORKING_DIR}/outputs', exist_ok=True)
print(f"Working directory created: {WORKING_DIR}")
```

```
Mounted at /content/drive
Working directory created: /content/drive/MyDrive/GenAI_Lab_Week10
```

```python
!pip install -q transformers datasets tokenizers tensorflow-addons
print("Libraries installed successfully!")
```

```
ERROR: Could not find a version that satisfies the requirement
tensorflow-addons (from versions: none)
ERROR: No matching distribution found for tensorflow-addons
Libraries installed successfully!
```

```python
from google.colab import drive
drive.mount('/content/drive')

# Create working directory
import os
WORKING_DIR = '/content/drive/MyDrive/GenAI_Lab_Week10_PyTorch'
os.makedirs(WORKING_DIR, exist_ok=True)
os.makedirs(f'{WORKING_DIR}/models', exist_ok=True)
os.makedirs(f'{WORKING_DIR}/outputs', exist_ok=True)
print(f"Working directory created: {WORKING_DIR}")
```

```
Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).
Working directory created:
/content/drive/MyDrive/GenAI_Lab_Week10_PyTorch
```

```python
# Install PyTorch and related libraries
!pip install -q torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu118
!pip install -q transformers datasets tokenizers tqdm matplotlib
seaborn

print("PyTorch and libraries installed successfully!")
```

```
PyTorch and libraries installed successfully!
```

```python
import numpy as np
import torch
```

```python
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torch.nn.functional as F
from transformers import GPT2Tokenizer, GPT2LMHeadModel, GPT2Config,
Trainer, TrainingArguments
from tokenizers import Tokenizer, models, pre_tokenizers, trainers
import json
import pickle
import random
import re
from datetime import datetime
import matplotlib.pyplot as plt
from tqdm import tqdm
import math

# Set random seeds for reproducibility
np.random.seed(42)
torch.manual_seed(42)
random.seed(42)

# Check device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"PyTorch version: {torch.__version__}")
print(f"Device available: {device}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
```

```
PyTorch version: 2.10.0+cu128
Device available: cuda
GPU: Tesla T4
```

```python
# Dataset from the lab
text_data = """artificial intelligence systems learn patterns from
data.
sequence models process information step by step.
recurrent neural networks are useful for sequence prediction.
lstm networks handle long term dependencies.
deep learning models improve sequence learning.
generative models create new samples from learned patterns.
language models predict the next word in a sentence.
sequence generation is used in chatbots and assistants.
machine learning helps computers learn automatically.
training data improves model accuracy.
neural networks simulate human brain structures.
optimization algorithms improve learning efficiency.
technology is transforming modern education.
online learning platforms use artificial intelligence.
students benefit from intelligent tutoring systems.
automation improves productivity and decision making."""
```

```
sentences = [s.strip() for s in text_data.strip().split('\n') if
s.strip()]
print(f"Total sentences: {len(sentences)}")
for i, sent in enumerate(sentences):
    print(f"{i+1}. {sent}")
```

```
Total sentences: 16
1. artificial intelligence systems learn patterns from data.
2. sequence models process information step by step.
3. recurrent neural networks are useful for sequence prediction.
4. lstm networks handle long term dependencies.
5. deep learning models improve sequence learning.
6. generative models create new samples from learned patterns.
7. language models predict the next word in a sentence.
8. sequence generation is used in chatbots and assistants.
9. machine learning helps computers learn automatically.
10. training data improves model accuracy.
11. neural networks simulate human brain structures.
12. optimization algorithms improve learning efficiency.
13. technology is transforming modern education.
14. online learning platforms use artificial intelligence.
15. students benefit from intelligent tutoring systems.
16. automation improves productivity and decision making.
```

```
# Data augmentation to increase dataset size
def augment_data(sentences):
    augmented = []

    # Original sentences
    augmented.extend(sentences)

    # Add variations with synonyms and rephrasing
    variations = {
        "artificial intelligence": ["ai", "machine intelligence",
"computational intelligence"],
        "machine learning": ["ml", "automated learning",
"computational learning"],
        "deep learning": ["deep neural networks", "hierarchical
learning"],
        "neural networks": ["neural nets", "artificial neural
networks"],
        "recurrent neural networks": ["rnns", "recurrent nets"],
        "lstm networks": ["long short-term memory", "lstm cells"],
        "sequence models": ["sequential models", "sequence
processors"],
        "generative models": ["generative ai", "generation models"],
        "language models": ["nlp models", "text models"],
        "training data": ["dataset", "learning data"],
        "optimization algorithms": ["optimizers", "training
```

```
algorithms"],
        "intelligent tutoring systems": ["smart tutoring", "ai
tutors"]
    }

    for sent in sentences:
        for key, vals in variations.items():
            if key in sent:
                for val in vals:
                    new_sent = sent.replace(key, val)
                    augmented.append(new_sent)

    # Add sentence with different punctuation
    for sent in sentences:
        augmented.append(sent.replace('.', '!'))
        augmented.append(sent.replace('.', '?'))

    return list(set(augmented))  # Remove duplicates

augmented_sentences = augment_data(sentences)
print(f"Original: {len(sentences)} sentences")
print(f"Augmented: {len(augmented_sentences)} sentences")

# Save augmented data
with open(f'{WORKING_DIR}/augmented_data.txt', 'w') as f:
    f.write('\n'.join(augmented_sentences))
```

```
Original: 16 sentences
Augmented: 79 sentences
```

```
# Character-level tokenization
class CharTokenizer:
    def __init__(self):
        self.char2idx = {}
        self.idx2char = {}
        self.vocab_size = 0

    def fit(self, texts):
        chars = sorted(list(set(''.join(texts))))
        self.char2idx = {c: i+1 for i, c in enumerate(chars)}  # 0
reserved for padding
        self.char2idx['<PAD>'] = 0
        self.char2idx['<UNK>'] = len(self.char2idx)
        self.idx2char = {i: c for c, i in self.char2idx.items()}
        self.vocab_size = len(self.char2idx)
        return self

    def encode(self, text):
        return [self.char2idx.get(c, self.char2idx['<UNK>']) for c in
text]
```

```
    def decode(self, indices):
        return ''.join([self.idx2char.get(i, '') for i in indices if i
!= 0])

    def save(self, path):
        with open(path, 'wb') as f:
            pickle.dump({'char2idx': self.char2idx, 'idx2char':
self.idx2char}, f)


    def load(self, path):
        with open(path, 'rb') as f:
            data = pickle.load(f)
            self.char2idx = data['char2idx']
            self.idx2char = data['idx2char']
            self.vocab_size = len(self.char2idx)
        return self

# Create and fit tokenizer
char_tokenizer = CharTokenizer().fit(augmented_sentences)
char_tokenizer.save(f'{WORKING_DIR}/char_tokenizer.pkl')
print(f"Vocabulary size: {char_tokenizer.vocab_size}")
print(f"Sample encoding: {char_tokenizer.encode('hello')}")
print(f"Sample decoding:
{char_tokenizer.decode(char_tokenizer.encode('hello'))}")
```

```
Vocabulary size: 32
Sample encoding: [13, 10, 16, 16, 19]
Sample decoding: hello
```

```
class CharDataset(Dataset):
    def __init__(self, texts, tokenizer, seq_length=40, step=2):
        self.tokenizer = tokenizer
        self.seq_length = seq_length
        self.sequences = []
        self.targets = []

        for text in texts:
            encoded = tokenizer.encode(text)
            for i in range(0, len(encoded) - seq_length, step):
                self.sequences.append(encoded[i:i + seq_length])
                self.targets.append(encoded[i + seq_length])

        self.sequences = torch.tensor(self.sequences,
dtype=torch.long)
        self.targets = torch.tensor(self.targets, dtype=torch.long)

    def __len__(self):
        return len(self.sequences)
```

```
    def __getitem__(self, idx):
        return self.sequences[idx], self.targets[idx]

# Create dataset
SEQ_LENGTH = 40
STEP = 2

char_dataset = CharDataset(augmented_sentences, char_tokenizer,
SEQ_LENGTH, STEP)
char_dataloader = DataLoader(char_dataset, batch_size=32,
shuffle=True)

print(f"Total sequences: {len(char_dataset)}")
print(f"Sample input shape: {char_dataset[0][0].shape}")
print(f"Sample target: {char_dataset[0][1]}")
```

```
Total sequences: 440
Sample input shape: torch.Size([40])
Sample target: 22
```

```
class LSTMGenerator(nn.Module):
    def __init__(self, vocab_size, embedding_dim=128, hidden_dim=256,
num_layers=3, dropout=0.3):
        super(LSTMGenerator, self).__init__()
        self.hidden_dim = hidden_dim
        self.num_layers = num_layers

        # Embedding layer
        self.embedding = nn.Embedding(vocab_size, embedding_dim,
padding_idx=0)

        # LSTM layers
        self.lstm = nn.LSTM(
            embedding_dim,
            hidden_dim,
            num_layers,
            batch_first=True,
            dropout=dropout if num_layers > 1 else 0,
            bidirectional=False
        )

        # Batch normalization
        self.batch_norm = nn.BatchNorm1d(hidden_dim)

        # Fully connected layers
        self.fc1 = nn.Linear(hidden_dim, 256)
        self.dropout1 = nn.Dropout(dropout)
        self.fc2 = nn.Linear(256, 128)
        self.dropout2 = nn.Dropout(dropout/2)
        self.fc3 = nn.Linear(128, vocab_size)
```

```
    def forward(self, x):
        # x shape: (batch, seq_length)
        embedded = self.embedding(x)  # (batch, seq_length,
embedding_dim)

        # LSTM
        lstm_out, (hidden, cell) = self.lstm(embedded)  # lstm_out:
(batch, seq_length, hidden_dim)

        # Take the last output
        last_output = lstm_out[:, -1, :]  # (batch, hidden_dim)

        # Batch norm
        last_output = self.batch_norm(last_output)

        # Fully connected with ReLU
        out = F.relu(self.fc1(last_output))
        out = self.dropout1(out)
        out = F.relu(self.fc2(out))
        out = self.dropout2(out)
        out = self.fc3(out)  # (batch, vocab_size)

        return out

    def init_hidden(self, batch_size):
        return (torch.zeros(self.num_layers, batch_size,
self.hidden_dim).to(device),
                torch.zeros(self.num_layers, batch_size,
self.hidden_dim).to(device))

# Initialize model
lstm_model = LSTMGenerator(
    vocab_size=char_tokenizer.vocab_size,
    embedding_dim=128,
    hidden_dim=256,
    num_layers=3,
    dropout=0.3
).to(device)

# Loss and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(lstm_model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer,
mode='min', factor=0.5, patience=5)

print(lstm_model)
print(f"Total parameters: {sum(p.numel() for p in
lstm_model.parameters()):,}")
```

```
LSTMGenerator(
  (embedding): Embedding(32, 128, padding_idx=0)
  (lstm): LSTM(128, 256, num_layers=3, batch_first=True, dropout=0.3)
  (batch_norm): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  (fc1): Linear(in_features=256, out_features=256, bias=True)
  (dropout1): Dropout(p=0.3, inplace=False)
  (fc2): Linear(in_features=256, out_features=128, bias=True)
  (dropout2): Dropout(p=0.15, inplace=False)
  (fc3): Linear(in_features=128, out_features=32, bias=True)
)
Total parameters: 1,555,360
```

```
def train_lstm(model, dataloader, criterion, optimizer, scheduler,
num_epochs=200):
    model.train()
    best_loss = float('inf')
    history = {'train_loss': [], 'train_acc': []}

    for epoch in range(num_epochs):
        total_loss = 0
        correct = 0
        total = 0

        progress_bar = tqdm(dataloader, desc=f'Epoch
{epoch+1}/{num_epochs}')

        for batch_idx, (inputs, targets) in enumerate(progress_bar):
            inputs, targets = inputs.to(device), targets.to(device)

            # Zero gradients
            optimizer.zero_grad()

            # Forward pass
            outputs = model(inputs)
            loss = criterion(outputs, targets)

            # Backward pass
            loss.backward()

            # Gradient clipping
            torch.nn.utils.clip_grad_norm_(model.parameters(),
max_norm=1.0)

            # Update weights
            optimizer.step()

            # Statistics
            total_loss += loss.item()
            _, predicted = outputs.max(1)
```

```
            total += targets.size(0)
            correct += predicted.eq(targets).sum().item()

            # Update progress bar
            progress_bar.set_postfix({
                'loss': f'{loss.item():.4f}',
                'acc': f'{100.*correct/total:.2f}%'
            })

        avg_loss = total_loss / len(dataloader)
        accuracy = 100. * correct / total

        history['train_loss'].append(avg_loss)
        history['train_acc'].append(accuracy)

        # Learning rate scheduling
        scheduler.step(avg_loss)

        # Save best model
        if avg_loss < best_loss:
            best_loss = avg_loss
            torch.save({
                'epoch': epoch,
                'model_state_dict': model.state_dict(),
                'optimizer_state_dict': optimizer.state_dict(),
                'loss': avg_loss,
            }, f'{WORKING_DIR}/models/lstm_best_model.pth')
            print(f"\n Saved best model with loss: {avg_loss:.4f}")

        if (epoch + 1) % 10 == 0:
            print(f"\nEpoch {epoch+1}: Loss = {avg_loss:.4f}, Accuracy
= {accuracy:.2f}%")

    # Save final model
    torch.save({
        'epoch': num_epochs,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'history': history,
    }, f'{WORKING_DIR}/models/lstm_final_model.pth')

    # Save history
    with open(f'{WORKING_DIR}/lstm_history.pkl', 'wb') as f:
        pickle.dump(history, f)

    return history

# Train the model
print("Starting LSTM training...")
lstm_history = train_lstm(lstm_model, char_dataloader, criterion,
```

```
optimizer, scheduler, num_epochs=200)
print("LSTM training completed!")
```

```
Starting LSTM training...

Epoch 1/200: 100%|██████████| 14/14 [00:01<00:00,  9.24it/s,
loss=2.7140, acc=13.64%]


 Saved best model with loss: 3.1895

Epoch 2/200: 100%|██████████| 14/14 [00:00<00:00, 71.81it/s,
loss=2.5849, acc=24.55%]


 Saved best model with loss: 2.5972

Epoch 3/200: 100%|██████████| 14/14 [00:00<00:00, 76.02it/s,
loss=2.2285, acc=35.23%]


 Saved best model with loss: 2.2538

Epoch 4/200: 100%|██████████| 14/14 [00:00<00:00, 71.21it/s,
loss=2.0340, acc=40.23%]


 Saved best model with loss: 1.9792

Epoch 5/200: 100%|██████████| 14/14 [00:00<00:00, 71.26it/s,
loss=2.0148, acc=53.64%]


 Saved best model with loss: 1.6366

Epoch 6/200: 100%|██████████| 14/14 [00:00<00:00, 72.28it/s,
loss=1.5047, acc=58.41%]


 Saved best model with loss: 1.4435

Epoch 7/200: 100%|██████████| 14/14 [00:00<00:00, 62.94it/s,
loss=1.3457, acc=62.73%]


 Saved best model with loss: 1.2282

Epoch 8/200: 100%|██████████| 14/14 [00:00<00:00, 64.02it/s,
loss=1.0211, acc=68.86%]


 Saved best model with loss: 0.9932
```

```
Epoch 9/200: 100%|██████████| 14/14 [00:00<00:00, 67.76it/s,
loss=0.9526, acc=73.64%]


 Saved best model with loss: 0.8581

Epoch 10/200: 100%|██████████| 14/14 [00:00<00:00, 84.16it/s,
loss=0.6504, acc=77.95%]


 Saved best model with loss: 0.6888

Epoch 10: Loss = 0.6888, Accuracy = 77.95%

Epoch 11/200: 100%|██████████| 14/14 [00:00<00:00, 97.50it/s,
loss=0.5027, acc=77.50%]


 Saved best model with loss: 0.6654

Epoch 12/200: 100%|██████████| 14/14 [00:00<00:00, 91.01it/s,
loss=0.6550, acc=84.55%]


 Saved best model with loss: 0.5244

Epoch 13/200: 100%|██████████| 14/14 [00:00<00:00, 93.53it/s,
loss=0.3417, acc=84.32%]


 Saved best model with loss: 0.5060

Epoch 14/200: 100%|██████████| 14/14 [00:00<00:00, 92.11it/s,
loss=0.7757, acc=83.86%]


 Saved best model with loss: 0.4966

Epoch 15/200: 100%|██████████| 14/14 [00:00<00:00, 93.81it/s,
loss=0.3394, acc=84.77%]


 Saved best model with loss: 0.4017

Epoch 16/200: 100%|██████████| 14/14 [00:00<00:00, 95.41it/s,
loss=0.3237, acc=89.09%]


 Saved best model with loss: 0.3430

Epoch 17/200: 100%|██████████| 14/14 [00:00<00:00, 95.37it/s,
loss=0.3048, acc=89.55%]
```

```
 Saved best model with loss: 0.2974

Epoch 18/200: 100%|██████████| 14/14 [00:00<00:00, 94.29it/s,
loss=0.2887, acc=88.18%]
Epoch 19/200: 100%|██████████| 14/14 [00:00<00:00, 99.72it/s,
loss=0.4011, acc=90.23%]


 Saved best model with loss: 0.2621

Epoch 20/200: 100%|██████████| 14/14 [00:00<00:00, 92.09it/s,
loss=0.2397, acc=91.36%]


 Saved best model with loss: 0.2400

Epoch 20: Loss = 0.2400, Accuracy = 91.36%

Epoch 21/200: 100%|██████████| 14/14 [00:00<00:00, 94.46it/s,
loss=0.2549, acc=90.91%]
Epoch 22/200: 100%|██████████| 14/14 [00:00<00:00, 100.85it/s,
loss=0.3301, acc=91.14%]
Epoch 23/200: 100%|██████████| 14/14 [00:00<00:00, 101.02it/s,
loss=0.1015, acc=91.59%]


 Saved best model with loss: 0.2106

Epoch 24/200: 100%|██████████| 14/14 [00:00<00:00, 90.44it/s,
loss=0.1869, acc=92.95%]


 Saved best model with loss: 0.1966

Epoch 25/200: 100%|██████████| 14/14 [00:00<00:00, 93.95it/s,
loss=0.4881, acc=91.59%]
Epoch 26/200: 100%|██████████| 14/14 [00:00<00:00, 90.60it/s,
loss=0.4204, acc=91.14%]
Epoch 27/200: 100%|██████████| 14/14 [00:00<00:00, 101.72it/s,
loss=0.0845, acc=92.27%]


 Saved best model with loss: 0.1924

Epoch 28/200: 100%|██████████| 14/14 [00:00<00:00, 93.96it/s,
loss=0.2833, acc=93.18%]


 Saved best model with loss: 0.1679

Epoch 29/200: 100%|██████████| 14/14 [00:00<00:00, 94.28it/s,
loss=0.0782, acc=93.41%]
```

```
 Saved best model with loss: 0.1531

Epoch 30/200: 100%|██████████| 14/14 [00:00<00:00, 89.78it/s,
loss=0.1039, acc=92.95%]


Epoch 30: Loss = 0.1578, Accuracy = 92.95%

Epoch 31/200: 100%|██████████| 14/14 [00:00<00:00, 100.39it/s,
loss=0.2784, acc=95.23%]


 Saved best model with loss: 0.1431

Epoch 32/200: 100%|██████████| 14/14 [00:00<00:00, 92.70it/s,
loss=0.0781, acc=93.18%]
Epoch 33/200: 100%|██████████| 14/14 [00:00<00:00, 98.29it/s,
loss=0.1719, acc=93.64%]
Epoch 34/200: 100%|██████████| 14/14 [00:00<00:00, 24.54it/s,
loss=0.1273, acc=94.09%]


 Saved best model with loss: 0.1385

Epoch 35/200: 100%|██████████| 14/14 [00:00<00:00, 21.53it/s,
loss=0.4443, acc=94.77%]


 Saved best model with loss: 0.1348

Epoch 36/200: 100%|██████████| 14/14 [00:00<00:00, 26.21it/s,
loss=0.3534, acc=94.32%]
Epoch 37/200: 100%|██████████| 14/14 [00:00<00:00, 43.05it/s,
loss=0.0707, acc=94.32%]


 Saved best model with loss: 0.1228

Epoch 38/200: 100%|██████████| 14/14 [00:00<00:00, 83.87it/s,
loss=0.1286, acc=95.00%]


 Saved best model with loss: 0.1227

Epoch 39/200: 100%|██████████| 14/14 [00:00<00:00, 94.28it/s,
loss=0.2432, acc=94.32%]
Epoch 40/200: 100%|██████████| 14/14 [00:00<00:00, 98.62it/s,
loss=0.0471, acc=93.86%]


Epoch 40: Loss = 0.1670, Accuracy = 93.86%
```

```
Epoch 41/200: 100%|██████████| 14/14 [00:00<00:00, 91.86it/s,
loss=0.1021, acc=94.32%]


 Saved best model with loss: 0.1174

Epoch 42/200: 100%|██████████| 14/14 [00:00<00:00, 96.05it/s,
loss=0.1630, acc=94.55%]
Epoch 43/200: 100%|██████████| 14/14 [00:00<00:00, 103.88it/s,
loss=0.0667, acc=95.45%]


 Saved best model with loss: 0.0963

Epoch 44/200: 100%|██████████| 14/14 [00:00<00:00, 86.86it/s,
loss=0.0898, acc=95.91%]


 Saved best model with loss: 0.0946

Epoch 45/200: 100%|██████████| 14/14 [00:00<00:00, 86.16it/s,
loss=0.1672, acc=95.23%]
Epoch 46/200: 100%|██████████| 14/14 [00:00<00:00, 96.91it/s,
loss=0.0594, acc=95.00%]
Epoch 47/200: 100%|██████████| 14/14 [00:00<00:00, 98.16it/s,
loss=0.1051, acc=95.91%]
Epoch 48/200: 100%|██████████| 14/14 [00:00<00:00, 99.10it/s,
loss=0.0982, acc=94.77%]
Epoch 49/200: 100%|██████████| 14/14 [00:00<00:00, 101.02it/s,
loss=0.3856, acc=95.00%]
Epoch 50/200: 100%|██████████| 14/14 [00:00<00:00, 88.89it/s,
loss=0.1541, acc=94.55%]


Epoch 50: Loss = 0.1378, Accuracy = 94.55%

Epoch 51/200: 100%|██████████| 14/14 [00:00<00:00, 102.50it/s,
loss=0.0181, acc=95.91%]
Epoch 52/200: 100%|██████████| 14/14 [00:00<00:00, 74.56it/s,
loss=0.0021, acc=95.45%]
Epoch 53/200: 100%|██████████| 14/14 [00:00<00:00, 64.59it/s,
loss=0.0261, acc=95.45%]
Epoch 54/200: 100%|██████████| 14/14 [00:00<00:00, 70.30it/s,
loss=0.1737, acc=94.55%]
Epoch 55/200: 100%|██████████| 14/14 [00:00<00:00, 73.95it/s,
loss=0.1264, acc=95.45%]
Epoch 56/200: 100%|██████████| 14/14 [00:00<00:00, 66.04it/s,
loss=0.0272, acc=96.14%]


 Saved best model with loss: 0.0848
```

```
Epoch 57/200: 100%|██████████| 14/14 [00:00<00:00, 76.67it/s,
loss=0.1866, acc=95.00%]
Epoch 58/200: 100%|██████████| 14/14 [00:00<00:00, 81.29it/s,
loss=0.1335, acc=95.00%]
Epoch 59/200: 100%|██████████| 14/14 [00:00<00:00, 79.67it/s,
loss=0.0444, acc=95.68%]
Epoch 60/200: 100%|██████████| 14/14 [00:00<00:00, 82.54it/s,
loss=0.0837, acc=95.68%]


Epoch 60: Loss = 0.1012, Accuracy = 95.68%

Epoch 61/200: 100%|██████████| 14/14 [00:00<00:00, 61.76it/s,
loss=0.1328, acc=95.68%]
Epoch 62/200: 100%|██████████| 14/14 [00:00<00:00, 71.40it/s,
loss=0.1104, acc=95.91%]
Epoch 63/200: 100%|██████████| 14/14 [00:00<00:00, 71.04it/s,
loss=0.1104, acc=95.68%]
Epoch 64/200: 100%|██████████| 14/14 [00:00<00:00, 70.51it/s,
loss=0.0975, acc=95.45%]
Epoch 65/200: 100%|██████████| 14/14 [00:00<00:00, 73.68it/s,
loss=0.1904, acc=96.36%]
Epoch 66/200: 100%|██████████| 14/14 [00:00<00:00, 69.27it/s,
loss=0.1232, acc=96.14%]
Epoch 67/200: 100%|██████████| 14/14 [00:00<00:00, 76.00it/s,
loss=0.0821, acc=95.91%]
Epoch 68/200: 100%|██████████| 14/14 [00:00<00:00, 76.67it/s,
loss=0.0039, acc=95.91%]


 Saved best model with loss: 0.0828

Epoch 69/200: 100%|██████████| 14/14 [00:00<00:00, 75.31it/s,
loss=0.0732, acc=95.23%]
Epoch 70/200: 100%|██████████| 14/14 [00:00<00:00, 74.98it/s,
loss=0.1142, acc=95.23%]


Epoch 70: Loss = 0.0895, Accuracy = 95.23%

Epoch 71/200: 100%|██████████| 14/14 [00:00<00:00, 63.98it/s,
loss=0.1574, acc=96.82%]
Epoch 72/200: 100%|██████████| 14/14 [00:00<00:00, 61.10it/s,
loss=0.0313, acc=95.68%]
Epoch 73/200: 100%|██████████| 14/14 [00:00<00:00, 67.48it/s,
loss=0.0880, acc=95.23%]
Epoch 74/200: 100%|██████████| 14/14 [00:00<00:00, 58.26it/s,
loss=0.1310, acc=95.68%]
Epoch 75/200: 100%|██████████| 14/14 [00:00<00:00, 88.91it/s,
loss=0.1414, acc=96.14%]
```

```
 Saved best model with loss: 0.0808

Epoch 76/200: 100%|██████████| 14/14 [00:00<00:00, 94.75it/s,
loss=0.0378, acc=95.68%]
Epoch 77/200: 100%|██████████| 14/14 [00:00<00:00, 89.02it/s,
loss=0.0534, acc=95.45%]
Epoch 78/200: 100%|██████████| 14/14 [00:00<00:00, 100.59it/s,
loss=0.0388, acc=95.91%]
Epoch 79/200: 100%|██████████| 14/14 [00:00<00:00, 102.05it/s,
loss=0.1565, acc=96.14%]
Epoch 80/200: 100%|██████████| 14/14 [00:00<00:00, 99.33it/s,
loss=0.0772, acc=95.45%]


Epoch 80: Loss = 0.0828, Accuracy = 95.45%

Epoch 81/200: 100%|██████████| 14/14 [00:00<00:00, 96.94it/s,
loss=0.0836, acc=96.36%]
Epoch 82/200: 100%|██████████| 14/14 [00:00<00:00, 78.31it/s,
loss=0.1050, acc=96.36%]
Epoch 83/200: 100%|██████████| 14/14 [00:00<00:00, 41.76it/s,
loss=0.0002, acc=95.45%]
Epoch 84/200: 100%|██████████| 14/14 [00:00<00:00, 44.34it/s,
loss=0.1073, acc=96.14%]
Epoch 85/200: 100%|██████████| 14/14 [00:00<00:00, 49.30it/s,
loss=0.1288, acc=95.68%]
Epoch 86/200: 100%|██████████| 14/14 [00:00<00:00, 37.80it/s,
loss=0.0687, acc=95.45%]
Epoch 87/200: 100%|██████████| 14/14 [00:00<00:00, 40.42it/s,
loss=0.0532, acc=95.45%]
Epoch 88/200: 100%|██████████| 14/14 [00:00<00:00, 95.37it/s,
loss=0.0703, acc=95.45%]
Epoch 89/200: 100%|██████████| 14/14 [00:00<00:00, 98.10it/s,
loss=0.0311, acc=95.91%]
Epoch 90/200: 100%|██████████| 14/14 [00:00<00:00, 94.95it/s,
loss=0.0706, acc=96.36%]


Epoch 90: Loss = 0.0823, Accuracy = 96.36%

Epoch 91/200: 100%|██████████| 14/14 [00:00<00:00, 94.77it/s,
loss=0.0509, acc=95.91%]
Epoch 92/200: 100%|██████████| 14/14 [00:00<00:00, 90.54it/s,
loss=0.0513, acc=95.91%]
Epoch 93/200: 100%|██████████| 14/14 [00:00<00:00, 98.95it/s,
loss=0.1050, acc=95.00%]
Epoch 94/200: 100%|██████████| 14/14 [00:00<00:00, 98.80it/s,
loss=0.0557, acc=95.91%]
Epoch 95/200: 100%|██████████| 14/14 [00:00<00:00, 94.74it/s,
loss=0.1405, acc=95.68%]
```

```
Epoch 96/200: 100%|██████████| 14/14 [00:00<00:00, 99.56it/s,
loss=0.0814, acc=95.91%]
Epoch 97/200: 100%|██████████| 14/14 [00:00<00:00, 91.21it/s,
loss=0.0015, acc=95.91%]


 Saved best model with loss: 0.0795

Epoch 98/200: 100%|██████████| 14/14 [00:00<00:00, 84.16it/s,
loss=0.0592, acc=95.68%]
Epoch 99/200: 100%|██████████| 14/14 [00:00<00:00, 94.01it/s,
loss=0.0026, acc=95.00%]
Epoch 100/200: 100%|██████████| 14/14 [00:00<00:00, 99.04it/s,
loss=0.0903, acc=95.45%]


Epoch 100: Loss = 0.0889, Accuracy = 95.45%

Epoch 101/200: 100%|██████████| 14/14 [00:00<00:00, 91.48it/s,
loss=0.0188, acc=95.68%]
Epoch 102/200: 100%|██████████| 14/14 [00:00<00:00, 98.27it/s,
loss=0.1238, acc=95.91%]
Epoch 103/200: 100%|██████████| 14/14 [00:00<00:00, 100.06it/s,
loss=0.0001, acc=96.59%]
Epoch 104/200: 100%|██████████| 14/14 [00:00<00:00, 96.34it/s,
loss=0.0653, acc=95.91%]
Epoch 105/200: 100%|██████████| 14/14 [00:00<00:00, 90.70it/s,
loss=0.0782, acc=95.45%]
Epoch 106/200: 100%|██████████| 14/14 [00:00<00:00, 100.80it/s,
loss=0.0561, acc=95.68%]
Epoch 107/200: 100%|██████████| 14/14 [00:00<00:00, 97.71it/s,
loss=0.0129, acc=95.23%]
Epoch 108/200: 100%|██████████| 14/14 [00:00<00:00, 95.19it/s,
loss=0.0518, acc=95.68%]
Epoch 109/200: 100%|██████████| 14/14 [00:00<00:00, 90.79it/s,
loss=0.1239, acc=96.36%]
Epoch 110/200: 100%|██████████| 14/14 [00:00<00:00, 94.80it/s,
loss=0.0701, acc=95.45%]


Epoch 110: Loss = 0.0850, Accuracy = 95.45%

Epoch 111/200: 100%|██████████| 14/14 [00:00<00:00, 96.88it/s,
loss=0.0471, acc=95.68%]
Epoch 112/200: 100%|██████████| 14/14 [00:00<00:00, 86.44it/s,
loss=0.0305, acc=96.59%]
Epoch 113/200: 100%|██████████| 14/14 [00:00<00:00, 97.95it/s,
loss=0.1033, acc=96.82%]
Epoch 114/200: 100%|██████████| 14/14 [00:00<00:00, 100.11it/s,
loss=0.0474, acc=95.91%]
Epoch 115/200: 100%|██████████| 14/14 [00:00<00:00, 95.15it/s,
```

```
loss=0.0237, acc=95.23%]
Epoch 116/200: 100%|██████████| 14/14 [00:00<00:00, 100.53it/s,
loss=0.0708, acc=95.91%]
Epoch 117/200: 100%|██████████| 14/14 [00:00<00:00, 96.00it/s,
loss=0.0248, acc=95.45%]
Epoch 118/200: 100%|██████████| 14/14 [00:00<00:00, 95.54it/s,
loss=0.0827, acc=96.36%]
Epoch 119/200: 100%|██████████| 14/14 [00:00<00:00, 88.13it/s,
loss=0.1118, acc=95.91%]
Epoch 120/200: 100%|██████████| 14/14 [00:00<00:00, 94.74it/s,
loss=0.0670, acc=95.91%]


Epoch 120: Loss = 0.0860, Accuracy = 95.91%

Epoch 121/200: 100%|██████████| 14/14 [00:00<00:00, 96.62it/s,
loss=0.1416, acc=95.68%]
Epoch 122/200: 100%|██████████| 14/14 [00:00<00:00, 97.51it/s,
loss=0.0001, acc=95.68%]
Epoch 123/200: 100%|██████████| 14/14 [00:00<00:00, 96.48it/s,
loss=0.0808, acc=95.68%]
Epoch 124/200: 100%|██████████| 14/14 [00:00<00:00, 97.99it/s,
loss=0.0272, acc=95.45%]
Epoch 125/200: 100%|██████████| 14/14 [00:00<00:00, 99.90it/s,
loss=0.0507, acc=95.68%]
Epoch 126/200: 100%|██████████| 14/14 [00:00<00:00, 86.25it/s,
loss=0.2120, acc=95.91%]
Epoch 127/200: 100%|██████████| 14/14 [00:00<00:00, 95.32it/s,
loss=0.1198, acc=95.45%]


 Saved best model with loss: 0.0783

Epoch 128/200: 100%|██████████| 14/14 [00:00<00:00, 85.14it/s,
loss=0.0944, acc=96.59%]
Epoch 129/200: 100%|██████████| 14/14 [00:00<00:00, 96.05it/s,
loss=0.1795, acc=95.91%]
Epoch 130/200: 100%|██████████| 14/14 [00:00<00:00, 89.70it/s,
loss=0.0582, acc=95.91%]


Epoch 130: Loss = 0.0851, Accuracy = 95.91%

Epoch 131/200: 100%|██████████| 14/14 [00:00<00:00, 89.26it/s,
loss=0.0948, acc=95.68%]
Epoch 132/200: 100%|██████████| 14/14 [00:00<00:00, 85.95it/s,
loss=0.0462, acc=95.45%]
Epoch 133/200: 100%|██████████| 14/14 [00:00<00:00, 86.46it/s,
loss=0.1474, acc=95.91%]
Epoch 134/200: 100%|██████████| 14/14 [00:00<00:00, 65.58it/s,
loss=0.0341, acc=95.91%]
```

```
Epoch 135/200: 100%|██████████| 14/14 [00:00<00:00, 65.50it/s,
loss=0.0820, acc=95.68%]
Epoch 136/200: 100%|██████████| 14/14 [00:00<00:00, 70.25it/s,
loss=0.0830, acc=95.23%]
Epoch 137/200: 100%|██████████| 14/14 [00:00<00:00, 65.09it/s,
loss=0.1003, acc=95.45%]
Epoch 138/200: 100%|██████████| 14/14 [00:00<00:00, 69.92it/s,
loss=0.1446, acc=95.91%]
Epoch 139/200: 100%|██████████| 14/14 [00:00<00:00, 76.85it/s,
loss=0.0548, acc=95.68%]
Epoch 140/200: 100%|██████████| 14/14 [00:00<00:00, 78.02it/s,
loss=0.0817, acc=96.36%]


Epoch 140: Loss = 0.0792, Accuracy = 96.36%

Epoch 141/200: 100%|██████████| 14/14 [00:00<00:00, 77.36it/s,
loss=0.0681, acc=96.14%]
Epoch 142/200: 100%|██████████| 14/14 [00:00<00:00, 78.65it/s,
loss=0.1551, acc=96.14%]
Epoch 143/200: 100%|██████████| 14/14 [00:00<00:00, 66.16it/s,
loss=0.0977, acc=95.91%]
Epoch 144/200: 100%|██████████| 14/14 [00:00<00:00, 63.69it/s,
loss=0.1843, acc=95.45%]
Epoch 145/200: 100%|██████████| 14/14 [00:00<00:00, 74.43it/s,
loss=0.1117, acc=95.91%]
Epoch 146/200: 100%|██████████| 14/14 [00:00<00:00, 74.64it/s,
loss=0.0294, acc=95.45%]
Epoch 147/200: 100%|██████████| 14/14 [00:00<00:00, 70.68it/s,
loss=0.0891, acc=96.36%]
Epoch 148/200: 100%|██████████| 14/14 [00:00<00:00, 70.18it/s,
loss=0.0574, acc=96.14%]
Epoch 149/200: 100%|██████████| 14/14 [00:00<00:00, 62.84it/s,
loss=0.1335, acc=95.91%]
Epoch 150/200: 100%|██████████| 14/14 [00:00<00:00, 72.68it/s,
loss=0.0283, acc=95.91%]


Epoch 150: Loss = 0.0811, Accuracy = 95.91%

Epoch 151/200: 100%|██████████| 14/14 [00:00<00:00, 72.37it/s,
loss=0.0476, acc=95.68%]
Epoch 152/200: 100%|██████████| 14/14 [00:00<00:00, 73.64it/s,
loss=0.0839, acc=96.36%]


 Saved best model with loss: 0.0769

Epoch 153/200: 100%|██████████| 14/14 [00:00<00:00, 58.73it/s,
loss=0.0949, acc=95.91%]
Epoch 154/200: 100%|██████████| 14/14 [00:00<00:00, 66.21it/s,
```

```
loss=0.0738, acc=95.91%]
Epoch 155/200: 100%|██████████| 14/14 [00:00<00:00, 59.11it/s,
loss=0.0745, acc=96.14%]
Epoch 156/200: 100%|██████████| 14/14 [00:00<00:00, 61.60it/s,
loss=0.1176, acc=95.68%]
Epoch 157/200: 100%|██████████| 14/14 [00:00<00:00, 95.30it/s,
loss=0.0887, acc=95.91%]
Epoch 158/200: 100%|██████████| 14/14 [00:00<00:00, 89.99it/s,
loss=0.1754, acc=95.91%]
Epoch 159/200: 100%|██████████| 14/14 [00:00<00:00, 93.71it/s,
loss=0.0591, acc=95.91%]
Epoch 160/200: 100%|██████████| 14/14 [00:00<00:00, 96.87it/s,
loss=0.0367, acc=95.68%]


Epoch 160: Loss = 0.0816, Accuracy = 95.68%

Epoch 161/200: 100%|██████████| 14/14 [00:00<00:00, 93.26it/s,
loss=0.0006, acc=96.36%]
Epoch 162/200: 100%|██████████| 14/14 [00:00<00:00, 100.31it/s,
loss=0.1253, acc=95.68%]
Epoch 163/200: 100%|██████████| 14/14 [00:00<00:00, 95.46it/s,
loss=0.0009, acc=95.68%]
Epoch 164/200: 100%|██████████| 14/14 [00:00<00:00, 99.73it/s,
loss=0.1176, acc=95.45%]
Epoch 165/200: 100%|██████████| 14/14 [00:00<00:00, 85.21it/s,
loss=0.0537, acc=95.91%]
Epoch 166/200: 100%|██████████| 14/14 [00:00<00:00, 91.25it/s,
loss=0.0557, acc=95.45%]
Epoch 167/200: 100%|██████████| 14/14 [00:00<00:00, 98.24it/s,
loss=0.0492, acc=95.68%]
Epoch 168/200: 100%|██████████| 14/14 [00:00<00:00, 90.41it/s,
loss=0.2054, acc=95.91%]
Epoch 169/200: 100%|██████████| 14/14 [00:00<00:00, 54.38it/s,
loss=0.1021, acc=95.91%]
Epoch 170/200: 100%|██████████| 14/14 [00:00<00:00, 54.62it/s,
loss=0.1059, acc=95.23%]


Epoch 170: Loss = 0.0891, Accuracy = 95.23%

Epoch 171/200: 100%|██████████| 14/14 [00:00<00:00, 29.47it/s,
loss=0.0674, acc=96.36%]
Epoch 172/200: 100%|██████████| 14/14 [00:00<00:00, 55.76it/s,
loss=0.0497, acc=95.00%]
Epoch 173/200: 100%|██████████| 14/14 [00:00<00:00, 52.23it/s,
loss=0.0930, acc=95.91%]
Epoch 174/200: 100%|██████████| 14/14 [00:00<00:00, 33.76it/s,
loss=0.0258, acc=96.14%]
Epoch 175/200: 100%|██████████| 14/14 [00:00<00:00, 62.79it/s,
```

```
loss=0.0020, acc=95.91%]
Epoch 176/200: 100%|██████████| 14/14 [00:00<00:00, 55.44it/s,
loss=0.0296, acc=95.00%]
Epoch 177/200: 100%|██████████| 14/14 [00:00<00:00, 46.29it/s,
loss=0.0625, acc=95.45%]
Epoch 178/200: 100%|██████████| 14/14 [00:00<00:00, 49.94it/s,
loss=0.0680, acc=96.14%]
Epoch 179/200: 100%|██████████| 14/14 [00:00<00:00, 53.16it/s,
loss=0.0238, acc=95.68%]
Epoch 180/200: 100%|██████████| 14/14 [00:00<00:00, 44.45it/s,
loss=0.0704, acc=96.14%]


Epoch 180: Loss = 0.0812, Accuracy = 96.14%

Epoch 181/200: 100%|██████████| 14/14 [00:00<00:00, 45.81it/s,
loss=0.0202, acc=95.45%]
Epoch 182/200: 100%|██████████| 14/14 [00:00<00:00, 51.55it/s,
loss=0.0877, acc=95.68%]
Epoch 183/200: 100%|██████████| 14/14 [00:00<00:00, 51.83it/s,
loss=0.0002, acc=95.68%]
Epoch 184/200: 100%|██████████| 14/14 [00:00<00:00, 96.11it/s,
loss=0.0671, acc=95.91%]
Epoch 185/200: 100%|██████████| 14/14 [00:00<00:00, 53.33it/s,
loss=0.0003, acc=96.36%]
Epoch 186/200: 100%|██████████| 14/14 [00:00<00:00, 44.14it/s,
loss=0.0542, acc=95.91%]
Epoch 187/200: 100%|██████████| 14/14 [00:00<00:00, 47.23it/s,
loss=0.0814, acc=95.68%]
Epoch 188/200: 100%|██████████| 14/14 [00:00<00:00, 48.63it/s,
loss=0.1864, acc=95.45%]
Epoch 189/200: 100%|██████████| 14/14 [00:00<00:00, 90.30it/s,
loss=0.1172, acc=96.36%]
Epoch 190/200: 100%|██████████| 14/14 [00:00<00:00, 97.67it/s,
loss=0.0260, acc=96.14%]


Epoch 190: Loss = 0.0814, Accuracy = 96.14%

Epoch 191/200: 100%|██████████| 14/14 [00:00<00:00, 87.34it/s,
loss=0.0003, acc=95.91%]
Epoch 192/200: 100%|██████████| 14/14 [00:00<00:00, 87.97it/s,
loss=0.0358, acc=95.91%]
Epoch 193/200: 100%|██████████| 14/14 [00:00<00:00, 98.26it/s,
loss=0.0894, acc=96.14%]
Epoch 194/200: 100%|██████████| 14/14 [00:00<00:00, 85.84it/s,
loss=0.1102, acc=95.68%]
Epoch 195/200: 100%|██████████| 14/14 [00:00<00:00, 88.48it/s,
loss=0.1527, acc=96.59%]
Epoch 196/200: 100%|██████████| 14/14 [00:00<00:00, 98.58it/s,
```

```
loss=0.0375, acc=95.68%]
Epoch 197/200: 100%|██████████| 14/14 [00:00<00:00, 95.85it/s,
loss=0.1311, acc=95.23%]
Epoch 198/200: 100%|██████████| 14/14 [00:00<00:00, 101.70it/s,
loss=0.0539, acc=95.23%]
Epoch 199/200: 100%|██████████| 14/14 [00:00<00:00, 92.55it/s,
loss=0.1233, acc=95.91%]
Epoch 200/200: 100%|██████████| 14/14 [00:00<00:00, 95.67it/s,
loss=0.1401, acc=95.91%]


Epoch 200: Loss = 0.0871, Accuracy = 95.91%
LSTM training completed!
```

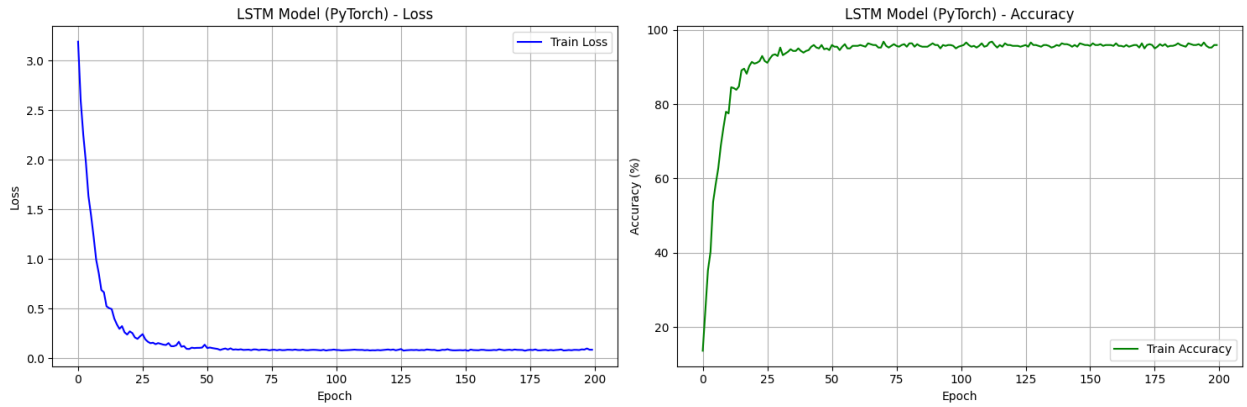
```python
def plot_history(history, title, save_path):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

    # Loss plot
    ax1.plot(history['train_loss'], label='Train Loss', color='blue')
    ax1.set_title(f'{title} - Loss')
    ax1.set_xlabel('Epoch')
    ax1.set_ylabel('Loss')
    ax1.legend()
    ax1.grid(True)

    # Accuracy plot
    ax2.plot(history['train_acc'], label='Train Accuracy',
color='green')
    ax2.set_title(f'{title} - Accuracy')
    ax2.set_xlabel('Epoch')
    ax2.set_ylabel('Accuracy (%)')
    ax2.legend()
    ax2.grid(True)

    plt.tight_layout()
    plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()
    print(f"Plot saved to: {save_path}")

plot_history(lstm_history, 'LSTM Model (PyTorch)',
f'{WORKING_DIR}/outputs/lstm_training_history.png')
```




```
Plot saved to:
/content/drive/MyDrive/GenAI_Lab_Week10_PyTorch/outputs/lstm_training_
history.png

def generate_text_lstm(model, tokenizer, seed_text, length=200,
temperature=0.8):
    model.eval()
    generated = seed_text
    encoded = tokenizer.encode(seed_text)

    with torch.no_grad():
        for _ in range(length):
            # Prepare input
            x = torch.tensor([encoded[-SEQ_LENGTH:]],
dtype=torch.long).to(device)

            # Predict
            output = model(x)

            # Apply temperature
            output = output / temperature
            probs = F.softmax(output, dim=1)

            # Sample
            next_char_idx = torch.multinomial(probs, 1).item()

            # Append to generated text
            next_char = tokenizer.idx2char.get(next_char_idx, '')
            generated += next_char
            encoded.append(next_char_idx)

    return generated

# Load best model for generation
checkpoint = torch.load(f'{WORKING_DIR}/models/lstm_best_model.pth')
lstm_model.load_state_dict(checkpoint['model_state_dict'])
lstm_model.eval()
```

```python
# Test generation
seed_texts = [
    "artificial intelligence",
    "machine learning",
    "deep learning",
    "neural networks",
    "sequence generation"
]

temperatures = [0.5, 0.7, 0.9]

print("="*60)
print("LSTM GENERATED TEXT (PyTorch)")
print("="*60)

lstm_results = {}
for seed in seed_texts:
    print(f"\nSeed: '{seed}'")
    print("-" * 60)
    for temp in temperatures:
        generated = generate_text_lstm(lstm_model, char_tokenizer,
seed, length=150, temperature=temp)
        print(f"\nTemperature {temp}:")
        print(generated)
        lstm_results[f"{seed}_temp{temp}"] = generated

# Save results
with open(f'{WORKING_DIR}/outputs/lstm_generated_text.txt', 'w') as f:
    for key, value in lstm_results.items():
        f.write(f"\n{'='*60}\n{key}\n{'='*60}\n{value}\n")
```

```
============================================================
LSTM GENERATED TEXT (PyTorch)
============================================================

Seed: 'artificial intelligence'
------------------------------------------------------------

Temperature 0.5:
artificial intelligence..epce..es..ence...rns from data..?a?ll..r
sequence
prediction..lligence..es..ence...eacin...ligence..es..ence...eacing..?
.in....in fr sequence predic

Temperature 0.7:
artificial intelligence...eacin..!..in syftion! learning....r
sefructures..frm data..!.a?lin.!?in!..in...in frm data.!.a.n.?a.in.!?
in....ins..ymtion! sefincy seqtence...es..

Temperature 0.9:
```

```
artificial intelligence...rns..frm data.??s.!...?s from data.!?a?
in!..liys.ence...rns from data!!!a.in!lli.rns..ru from data..?a?lli!
s..fr sequence prediction?.llirn sefince

Seed: 'machine learning'
--------------------------------------------------------------

Temperature 0.5:
machine learning.....rn sefince learning.???ion..lliern sefrn
uromdicta?ll..r sequence prediction...lion sequence prediction!!
lliing...!.es.ce....an.in...incies..frm

Temperature 0.7:
machine learning..!.cies..frm data.??a...a!?in.?
ligence...es..ence...rns.frm data..?!a!in!!.in...in.. from data!!!a!
in!.licies..frm data!?!a.in.!licies..frm data???a?

Temperature 0.9:
machine learning..?.!in.!.ligence...rns from data.!..a.!a!
in?.ligence..es..ence...rns. from data..?a.?a!?ains.?!a!in!?!
in....ins..fimn. fro sequence prediction.!!liin

Seed: 'deep learning'
--------------------------------------------------------------

Temperature 0.5:
deep learning!...frn sefincy sefince.ligence...rns..frm data.!.!a.in!?
licion..lirn sefince learning.?...rn sefince learning.....rn sefince
learning.!!...rns from d

Temperature 0.7:
deep learning.!..fin fr sequence
prediction..lligence..pes.ence...eacin....rn sefince learning.....r
sefrn urom data.!.a?lin...a.ing!!...es..es..em learn batancios

Temperature 0.9:
deep learning?..nfies..ymm!..?a.n!ins.!.. sequence
prediction..lligence..rediction.. sequence prediction...llir sequence
prediction!..linf.in fr sequence predictio

Seed: 'neural networks'
--------------------------------------------------------------

Temperature 0.5:
neural networks seffllfr sequence
prediction!.lligence..pes.ence...eacing....rn sefince learning.!...rns
from data?.a.!.a.n.a.ing.!...rns from data!!.a!.la.!in.!.in.

Temperature 0.7:
neural networks seffllfr sequence prediction...llir sequence
```

```
prediction..lligence..res..rm data...!?a!in.!.ligence...res..frm
data?..a!?lion..laince...rndin.? fro se

Temperature 0.9:
neural networks syfmm data!?.a..!a.lin...in..!
incies..yfmin?.lligence...res..frm data?!.a?in..ligence...rn...r
sefouing fimn!y?in...?in! frm data?!?a.!in..ligence...

Seed: 'sequence generation'
------------------------------------------------------------

Temperature 0.5:
sequence generation!!liing ytemncies..ge from learndda!ption..llirn
sefince learning??..rns.frm data?!?a?in!..licres..frm data!?.a..a?
in.!..in....in fr sequence predicti

Temperature 0.7:
sequence generation.?limn fron sequence prediction!.llifrn sefrnc
rediction..?lience prediction?.llirn sefrnc fre datin.....in. frin frm
data!..a.lin!!?in.?ligence...red

Temperature 0.9:
sequence generation...lin fr sequence prediction?.llir sequence
prediction??llires. from data!?a?lin.?.lirns.frm data.!?-
a.in?.ligence...es..ence...eacing..!..rns from d
```

```python
# Train word-level tokenizer
def train_word_tokenizer(texts, vocab_size=500):
    tokenizer = Tokenizer(models.WordPiece(unk_token="[UNK]"))
    tokenizer.pre_tokenizer = pre_tokenizers.Whitespace()

    trainer = trainers.WordPieceTrainer(
        vocab_size=vocab_size,
        special_tokens=["[PAD]", "[UNK]", "[CLS]", "[SEP]", "[MASK]"]
    )

    tokenizer.train_from_iterator(texts, trainer=trainer)
    return tokenizer

word_tokenizer = train_word_tokenizer(augmented_sentences,
vocab_size=300)
word_tokenizer.save(f'{WORKING_DIR}/word_tokenizer.json')

# Test tokenizer
test_text = "artificial intelligence systems"
encoding = word_tokenizer.encode(test_text)
print(f"Original: {test_text}")
print(f"Tokens: {encoding.tokens}")
print(f"IDs: {encoding.ids}")
```

```
Original: artificial intelligence systems
Tokens: ['artificial', 'intelligence', 'systems']
IDs: [163, 144, 155]
```

```python
class WordDataset(Dataset):
    def __init__(self, texts, tokenizer, max_length=40):
        self.tokenizer = tokenizer
        self.max_length = max_length
        self.vocab_size = tokenizer.get_vocab_size()

        self.input_ids = []
        self.target_ids = []

        for text in texts:
            encoding = tokenizer.encode(text)
            ids = encoding.ids[:max_length]

            # Create input-target pairs (shifted by 1)
            for i in range(1, len(ids)):
                input_seq = ids[:i] + [0] * (max_length - i)
                target_seq = ids[1:i+1] + [0] * (max_length - i)

                self.input_ids.append(input_seq)
                self.target_ids.append(target_seq)

        self.input_ids = torch.tensor(self.input_ids,
dtype=torch.long)
        self.target_ids = torch.tensor(self.target_ids,
dtype=torch.long)

    def __len__(self):
        return len(self.input_ids)

    def __getitem__(self, idx):
        return self.input_ids[idx], self.target_ids[idx]

# Create dataset
word_dataset = WordDataset(augmented_sentences, word_tokenizer,
max_length=40)
word_dataloader = DataLoader(word_dataset, batch_size=16,
shuffle=True)

print(f"Transformer dataset size: {len(word_dataset)}")
print(f"Sample input shape: {word_dataset[0][0].shape}")
print(f"Sample target shape: {word_dataset[0][1].shape}")
```

```
Transformer dataset size: 696
Sample input shape: torch.Size([40])
Sample target shape: torch.Size([40])
```

```python
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super().__init__()

        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len,
dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-
math.log(10000.0) / d_model))

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

        pe = pe.unsqueeze(0)
        self.register_buffer('pe', pe)

    def forward(self, x):
        return x + self.pe[:, :x.size(1), :]

class TransformerBlock(nn.Module):
    def __init__(self, d_model, num_heads, ff_dim, dropout=0.1):
        super().__init__()

        self.attention = nn.MultiheadAttention(d_model, num_heads,
dropout=dropout, batch_first=True)
        self.norm1 = nn.LayerNorm(d_model)

        self.ff = nn.Sequential(
            nn.Linear(d_model, ff_dim),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(ff_dim, d_model)
        )
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    # Modified to accept attn_mask
    def forward(self, x, attn_mask=None):
        # Self-attention with causal mask
        attn_output, _ = self.attention(x, x, x, attn_mask=attn_mask,
is_causal=True)
        x = self.norm1(x + self.dropout(attn_output))

        # Feed forward
        ff_output = self.ff(x)
        x = self.norm2(x + self.dropout(ff_output))

        return x

class TransformerGenerator(nn.Module):
    def __init__(self, vocab_size, max_len=40, d_model=128,
```

```
num_heads=4, ff_dim=512, num_layers=3, dropout=0.1):
        super().__init__()

        self.embedding = nn.Embedding(vocab_size, d_model)
        self.pos_encoding = PositionalEncoding(d_model, max_len)
        self.dropout = nn.Dropout(dropout)

        self.transformer_blocks = nn.ModuleList([
            TransformerBlock(d_model, num_heads, ff_dim, dropout)
            for _ in range(num_layers)
        ])

        self.fc = nn.Linear(d_model, vocab_size)
        self.max_len = max_len # Store max_len for mask generation

    def forward(self, x):
        # x shape: (batch, seq_len)
        seq_len = x.size(1)

        x = self.embedding(x)  # (batch, seq_len, d_model)
        x = self.pos_encoding(x)
        x = self.dropout(x)

        # Generate a square subsequent mask for the attention
mechanism
        # This mask prevents tokens from attending to future tokens
(causal attention).
        # It's typically a square matrix with -inf in the upper
triangle.
        # The MultiheadAttention module expects it as a float tensor.
        attn_mask =
torch.nn.Transformer.generate_square_subsequent_mask(seq_len).to(x.dev
ice)

        # Apply transformer blocks, passing the causal mask
        for block in self.transformer_blocks:
            x = block(x, attn_mask=attn_mask) # Pass the generated
mask

        # Output
        output = self.fc(x)  # (batch, seq_len, vocab_size)
        return output

# Initialize model
transformer_model = TransformerGenerator(
    vocab_size=word_tokenizer.get_vocab_size(),
    max_len=40,
    d_model=128,
    num_heads=4,
    ff_dim=512,
    num_layers=3,
```

```
    dropout=0.1
).to(device)

# Loss and optimizer (ignore padding index 0)
criterion_trans = nn.CrossEntropyLoss(ignore_index=0)
optimizer_trans = optim.Adam(transformer_model.parameters(),
lr=0.0001)
scheduler_trans =
optim.lr_scheduler.ReduceLROnPlateau(optimizer_trans, mode='min',
factor=0.5, patience=7)

print(transformer_model)
print(f"Total parameters: {sum(p.numel() for p in
transformer_model.parameters()):,}")
```

```
TransformerGenerator(
  (embedding): Embedding(300, 128)
  (pos_encoding): PositionalEncoding()
  (dropout): Dropout(p=0.1, inplace=False)
  (transformer_blocks): ModuleList(
    (0-2): 3 x TransformerBlock(
      (attention): MultiheadAttention(
        (out_proj): NonDynamicallyQuantizableLinear(in_features=128,
out_features=128, bias=True)
      )
      (norm1): LayerNorm((128,), eps=1e-05, elementwise_affine=True)
      (ff): Sequential(
        (0): Linear(in_features=128, out_features=512, bias=True)
        (1): ReLU()
        (2): Dropout(p=0.1, inplace=False)
        (3): Linear(in_features=512, out_features=128, bias=True)
      )
      (norm2): LayerNorm((128,), eps=1e-05, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
  )
  (fc): Linear(in_features=128, out_features=300, bias=True)
)
Total parameters: 671,916
```

```
def train_transformer(model, dataloader, criterion, optimizer,
scheduler, num_epochs=300):
    model.train()
    best_loss = float('inf')
    history = {'train_loss': [], 'train_acc': []}

    for epoch in range(num_epochs):
        total_loss = 0
        correct = 0
        total = 0
```

```
        progress_bar = tqdm(dataloader, desc=f'Epoch
{epoch+1}/{num_epochs}')

        for inputs, targets in progress_bar:
            inputs, targets = inputs.to(device), targets.to(device)

            # Zero gradients
            optimizer.zero_grad()

            # Forward pass
            outputs = model(inputs)  # (batch, seq_len, vocab_size)

            # Reshape for loss calculation
            outputs_flat = outputs.view(-1, outputs.size(-1))
            targets_flat = targets.view(-1)

            loss = criterion(outputs_flat, targets_flat)

            # Backward pass
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(),
max_norm=1.0)
            optimizer.step()

            # Statistics
            total_loss += loss.item()
            _, predicted = outputs_flat.max(1)
            mask = targets_flat != 0  # Ignore padding
            correct +=
predicted.eq(targets_flat).masked_select(mask).sum().item()
            total += mask.sum().item()

            progress_bar.set_postfix({
                'loss': f'{loss.item():.4f}',
                'acc': f'{100.*correct/total:.2f}%' if total > 0 else
'0%'
            })

        avg_loss = total_loss / len(dataloader)
        accuracy = 100. * correct / total if total > 0 else 0

        history['train_loss'].append(avg_loss)
        history['train_acc'].append(accuracy)

        scheduler.step(avg_loss)

        # Save best model
        if avg_loss < best_loss:
            best_loss = avg_loss
```

```
            torch.save({
                'epoch': epoch,
                'model_state_dict': model.state_dict(),
                'optimizer_state_dict': optimizer.state_dict(),
                'loss': avg_loss,
            }, f'{WORKING_DIR}/models/transformer_best_model.pth')
            print(f"\n□ Saved best model with loss: {avg_loss:.4f}")

        if (epoch + 1) % 10 == 0:
            print(f"\nEpoch {epoch+1}: Loss = {avg_loss:.4f}, Accuracy
= {accuracy:.2f}%")

    # Save final model
    torch.save({
        'epoch': num_epochs,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'history': history,
    }, f'{WORKING_DIR}/models/transformer_final_model.pth')

    with open(f'{WORKING_DIR}/transformer_history.pkl', 'wb') as f:
        pickle.dump(history, f)

    return history

print("Starting Transformer training...")
transformer_history = train_transformer(transformer_model,
word_dataloader, criterion_trans, optimizer_trans, scheduler_trans,
num_epochs=300)
print("Transformer training completed!")
```

```
Starting Transformer training...

Epoch 1/300: 100%|██████████| 44/44 [00:00<00:00, 75.43it/s,
loss=4.5763, acc=5.91%]


□ Saved best model with loss: 5.2392

Epoch 2/300: 100%|██████████| 44/44 [00:00<00:00, 80.83it/s,
loss=3.7227, acc=33.71%]


□ Saved best model with loss: 4.2017

Epoch 3/300: 100%|██████████| 44/44 [00:00<00:00, 65.37it/s,
loss=2.6168, acc=59.65%]


□ Saved best model with loss: 3.3572
```

```
Epoch 4/300: 100%|██████████| 44/44 [00:00<00:00, 63.78it/s,
loss=2.4345, acc=71.25%]


 Saved best model with loss: 2.6820

Epoch 5/300: 100%|██████████| 44/44 [00:01<00:00, 43.77it/s,
loss=1.8539, acc=77.45%]


 Saved best model with loss: 2.1512

Epoch 6/300: 100%|██████████| 44/44 [00:00<00:00, 63.47it/s,
loss=1.7137, acc=80.56%]


 Saved best model with loss: 1.7449

Epoch 7/300: 100%|██████████| 44/44 [00:00<00:00, 60.85it/s,
loss=1.1051, acc=83.83%]


 Saved best model with loss: 1.4316

Epoch 8/300: 100%|██████████| 44/44 [00:00<00:00, 61.83it/s,
loss=0.6483, acc=85.86%]


 Saved best model with loss: 1.1829

Epoch 9/300: 100%|██████████| 44/44 [00:00<00:00, 83.87it/s,
loss=1.0058, acc=88.28%]


 Saved best model with loss: 0.9895

Epoch 10/300: 100%|██████████| 44/44 [00:00<00:00, 88.84it/s,
loss=0.6913, acc=89.81%]


 Saved best model with loss: 0.8318

Epoch 10: Loss = 0.8318, Accuracy = 89.81%

Epoch 11/300: 100%|██████████| 44/44 [00:00<00:00, 84.17it/s,
loss=0.9097, acc=91.55%]


 Saved best model with loss: 0.7068

Epoch 12/300: 100%|██████████| 44/44 [00:00<00:00, 90.22it/s,
loss=0.6327, acc=92.70%]
```

```
 Saved best model with loss: 0.6070

Epoch 13/300: 100%|██████████| 44/44 [00:00<00:00, 77.90it/s,
loss=0.7804, acc=93.19%]


 Saved best model with loss: 0.5257

Epoch 14/300: 100%|██████████| 44/44 [00:00<00:00, 82.80it/s,
loss=0.5585, acc=94.04%]


 Saved best model with loss: 0.4644

Epoch 15/300: 100%|██████████| 44/44 [00:00<00:00, 82.33it/s,
loss=0.3148, acc=94.28%]


 Saved best model with loss: 0.4164

Epoch 16/300: 100%|██████████| 44/44 [00:00<00:00, 84.12it/s,
loss=0.5596, acc=94.26%]


 Saved best model with loss: 0.3742

Epoch 17/300: 100%|██████████| 44/44 [00:00<00:00, 85.45it/s,
loss=0.3644, acc=94.85%]


 Saved best model with loss: 0.3420

Epoch 18/300: 100%|██████████| 44/44 [00:00<00:00, 84.14it/s,
loss=0.4191, acc=94.72%]


 Saved best model with loss: 0.3118

Epoch 19/300: 100%|██████████| 44/44 [00:00<00:00, 82.10it/s,
loss=0.2767, acc=94.80%]


 Saved best model with loss: 0.2884

Epoch 20/300: 100%|██████████| 44/44 [00:00<00:00, 81.23it/s,
loss=0.2689, acc=94.80%]


 Saved best model with loss: 0.2682

Epoch 20: Loss = 0.2682, Accuracy = 94.80%
```

```
Epoch 21/300: 100%|██████████| 44/44 [00:00<00:00, 86.97it/s,
loss=0.1230, acc=94.92%]


 Saved best model with loss: 0.2560

Epoch 22/300: 100%|██████████| 44/44 [00:00<00:00, 85.72it/s,
loss=0.2753, acc=94.89%]


 Saved best model with loss: 0.2410

Epoch 23/300: 100%|██████████| 44/44 [00:00<00:00, 85.38it/s,
loss=0.2444, acc=95.02%]


 Saved best model with loss: 0.2330

Epoch 24/300: 100%|██████████| 44/44 [00:00<00:00, 82.74it/s,
loss=0.2050, acc=94.97%]


 Saved best model with loss: 0.2203

Epoch 25/300: 100%|██████████| 44/44 [00:00<00:00, 74.06it/s,
loss=0.1881, acc=94.99%]


 Saved best model with loss: 0.2111

Epoch 26/300: 100%|██████████| 44/44 [00:00<00:00, 67.19it/s,
loss=0.4559, acc=95.11%]


 Saved best model with loss: 0.2056

Epoch 27/300: 100%|██████████| 44/44 [00:00<00:00, 66.30it/s,
loss=0.1964, acc=95.09%]


 Saved best model with loss: 0.1953

Epoch 28/300: 100%|██████████| 44/44 [00:02<00:00, 19.55it/s,
loss=0.1576, acc=94.97%]


 Saved best model with loss: 0.1913

Epoch 29/300: 100%|██████████| 44/44 [00:01<00:00, 29.69it/s,
loss=0.1683, acc=95.11%]


 Saved best model with loss: 0.1871
```

```
Epoch 30/300: 100%|██████████| 44/44 [00:00<00:00, 83.19it/s,
loss=0.1090, acc=95.24%]


 Saved best model with loss: 0.1781

Epoch 30: Loss = 0.1781, Accuracy = 95.24%

Epoch 31/300: 100%|██████████| 44/44 [00:00<00:00, 84.43it/s,
loss=0.2058, acc=94.94%]


 Saved best model with loss: 0.1773

Epoch 32/300: 100%|██████████| 44/44 [00:01<00:00, 33.00it/s,
loss=0.1812, acc=95.04%]


 Saved best model with loss: 0.1753

Epoch 33/300: 100%|██████████| 44/44 [00:00<00:00, 77.00it/s,
loss=0.0940, acc=95.02%]


 Saved best model with loss: 0.1647

Epoch 34/300: 100%|██████████| 44/44 [00:00<00:00, 81.21it/s,
loss=0.1209, acc=95.11%]
Epoch 35/300: 100%|██████████| 44/44 [00:00<00:00, 80.23it/s,
loss=0.2284, acc=95.19%]


 Saved best model with loss: 0.1642

Epoch 36/300: 100%|██████████| 44/44 [00:00<00:00, 77.73it/s,
loss=0.1908, acc=95.09%]


 Saved best model with loss: 0.1631

Epoch 37/300: 100%|██████████| 44/44 [00:00<00:00, 74.25it/s,
loss=0.1665, acc=94.80%]
Epoch 38/300: 100%|██████████| 44/44 [00:00<00:00, 84.21it/s,
loss=0.3188, acc=94.87%]
Epoch 39/300: 100%|██████████| 44/44 [00:00<00:00, 83.95it/s,
loss=0.2893, acc=95.26%]


 Saved best model with loss: 0.1554

Epoch 40/300: 100%|██████████| 44/44 [00:00<00:00, 79.54it/s,
loss=0.2242, acc=95.09%]
```

```
Epoch 40: Loss = 0.1574, Accuracy = 95.09%

Epoch 41/300: 100%|██████████| 44/44 [00:00<00:00, 83.02it/s,
loss=0.1537, acc=95.11%]


 Saved best model with loss: 0.1489

Epoch 42/300: 100%|██████████| 44/44 [00:00<00:00, 83.53it/s,
loss=0.1193, acc=94.97%]
Epoch 43/300: 100%|██████████| 44/44 [00:00<00:00, 67.67it/s,
loss=0.1316, acc=95.16%]


 Saved best model with loss: 0.1447

Epoch 44/300: 100%|██████████| 44/44 [00:00<00:00, 64.51it/s,
loss=0.1440, acc=94.80%]
Epoch 45/300: 100%|██████████| 44/44 [00:00<00:00, 70.03it/s,
loss=0.1664, acc=94.99%]


 Saved best model with loss: 0.1447

Epoch 46/300: 100%|██████████| 44/44 [00:00<00:00, 62.59it/s,
loss=0.1158, acc=95.16%]


 Saved best model with loss: 0.1440

Epoch 47/300: 100%|██████████| 44/44 [00:00<00:00, 66.75it/s,
loss=0.1356, acc=95.26%]
Epoch 48/300: 100%|██████████| 44/44 [00:00<00:00, 60.45it/s,
loss=0.0997, acc=95.14%]


 Saved best model with loss: 0.1408

Epoch 49/300: 100%|██████████| 44/44 [00:00<00:00, 78.74it/s,
loss=0.0846, acc=95.02%]


 Saved best model with loss: 0.1377

Epoch 50/300: 100%|██████████| 44/44 [00:00<00:00, 81.23it/s,
loss=0.0805, acc=94.89%]


Epoch 50: Loss = 0.1408, Accuracy = 94.89%

Epoch 51/300: 100%|██████████| 44/44 [00:00<00:00, 85.78it/s,
loss=0.1772, acc=95.14%]
```

```
Epoch 52/300: 100%|██████████| 44/44 [00:00<00:00, 87.70it/s,
loss=0.0940, acc=94.97%]
Epoch 53/300: 100%|██████████| 44/44 [00:01<00:00, 41.54it/s,
loss=0.1342, acc=95.21%]


 Saved best model with loss: 0.1370

Epoch 54/300: 100%|██████████| 44/44 [00:00<00:00, 63.98it/s,
loss=0.0813, acc=95.26%]


 Saved best model with loss: 0.1358

Epoch 55/300: 100%|██████████| 44/44 [00:00<00:00, 80.25it/s,
loss=0.1727, acc=95.04%]


 Saved best model with loss: 0.1349

Epoch 56/300: 100%|██████████| 44/44 [00:00<00:00, 82.69it/s,
loss=0.0762, acc=94.92%]
Epoch 57/300: 100%|██████████| 44/44 [00:00<00:00, 82.67it/s,
loss=0.0438, acc=95.04%]
Epoch 58/300: 100%|██████████| 44/44 [00:00<00:00, 88.65it/s,
loss=0.1860, acc=95.02%]


 Saved best model with loss: 0.1346

Epoch 59/300: 100%|██████████| 44/44 [00:00<00:00, 81.88it/s,
loss=0.0889, acc=95.19%]


 Saved best model with loss: 0.1322

Epoch 60/300: 100%|██████████| 44/44 [00:00<00:00, 83.23it/s,
loss=0.2137, acc=95.02%]


Epoch 60: Loss = 0.1354, Accuracy = 95.02%

Epoch 61/300: 100%|██████████| 44/44 [00:00<00:00, 80.27it/s,
loss=0.1536, acc=95.02%]


 Saved best model with loss: 0.1317

Epoch 62/300: 100%|██████████| 44/44 [00:00<00:00, 81.88it/s,
loss=0.0899, acc=95.26%]
Epoch 63/300: 100%|██████████| 44/44 [00:00<00:00, 87.31it/s,
loss=0.1438, acc=95.43%]
```

```
 Saved best model with loss: 0.1282

Epoch 64/300: 100%|██████████| 44/44 [00:00<00:00, 85.33it/s,
loss=0.2578, acc=95.07%]
Epoch 65/300: 100%|██████████| 44/44 [00:00<00:00, 81.13it/s,
loss=0.2282, acc=94.99%]
Epoch 66/300: 100%|██████████| 44/44 [00:00<00:00, 61.61it/s,
loss=0.1427, acc=95.04%]
Epoch 67/300: 100%|██████████| 44/44 [00:00<00:00, 66.99it/s,
loss=0.0796, acc=95.07%]
Epoch 68/300: 100%|██████████| 44/44 [00:00<00:00, 62.71it/s,
loss=0.0934, acc=94.89%]
Epoch 69/300: 100%|██████████| 44/44 [00:00<00:00, 61.04it/s,
loss=0.1025, acc=95.09%]


 Saved best model with loss: 0.1268

Epoch 70/300: 100%|██████████| 44/44 [00:00<00:00, 66.64it/s,
loss=0.2171, acc=95.02%]


Epoch 70: Loss = 0.1304, Accuracy = 95.02%

Epoch 71/300: 100%|██████████| 44/44 [00:00<00:00, 55.51it/s,
loss=0.1756, acc=95.04%]
Epoch 72/300: 100%|██████████| 44/44 [00:00<00:00, 74.76it/s,
loss=0.0519, acc=95.14%]
Epoch 73/300: 100%|██████████| 44/44 [00:00<00:00, 78.72it/s,
loss=0.1421, acc=95.19%]
Epoch 74/300: 100%|██████████| 44/44 [00:00<00:00, 82.85it/s,
loss=0.0513, acc=95.07%]


 Saved best model with loss: 0.1265

Epoch 75/300: 100%|██████████| 44/44 [00:00<00:00, 81.64it/s,
loss=0.0853, acc=95.14%]
Epoch 76/300: 100%|██████████| 44/44 [00:00<00:00, 83.37it/s,
loss=0.1357, acc=95.19%]
Epoch 77/300: 100%|██████████| 44/44 [00:00<00:00, 84.78it/s,
loss=0.2342, acc=95.21%]


 Saved best model with loss: 0.1264

Epoch 78/300: 100%|██████████| 44/44 [00:00<00:00, 75.69it/s,
loss=0.0493, acc=94.97%]


 Saved best model with loss: 0.1258
```

```
Epoch 79/300: 100%|██████████| 44/44 [00:00<00:00, 71.48it/s,
loss=0.2142, acc=95.02%]
Epoch 80/300: 100%|██████████| 44/44 [00:00<00:00, 84.58it/s,
loss=0.0474, acc=95.19%]


 Saved best model with loss: 0.1248

Epoch 80: Loss = 0.1248, Accuracy = 95.19%

Epoch 81/300: 100%|██████████| 44/44 [00:00<00:00, 82.03it/s,
loss=0.1279, acc=95.14%]
Epoch 82/300: 100%|██████████| 44/44 [00:00<00:00, 83.02it/s,
loss=0.2278, acc=94.94%]
Epoch 83/300: 100%|██████████| 44/44 [00:00<00:00, 87.58it/s,
loss=0.0414, acc=95.11%]


 Saved best model with loss: 0.1234

Epoch 84/300: 100%|██████████| 44/44 [00:00<00:00, 82.21it/s,
loss=0.2434, acc=95.21%]
Epoch 85/300: 100%|██████████| 44/44 [00:00<00:00, 80.99it/s,
loss=0.2068, acc=95.19%]
Epoch 86/300: 100%|██████████| 44/44 [00:00<00:00, 84.89it/s,
loss=0.1033, acc=94.89%]
Epoch 87/300: 100%|██████████| 44/44 [00:00<00:00, 83.19it/s,
loss=0.1102, acc=95.33%]


 Saved best model with loss: 0.1221

Epoch 88/300: 100%|██████████| 44/44 [00:00<00:00, 81.89it/s,
loss=0.1655, acc=95.02%]
Epoch 89/300: 100%|██████████| 44/44 [00:00<00:00, 83.17it/s,
loss=0.1501, acc=94.99%]
Epoch 90/300: 100%|██████████| 44/44 [00:00<00:00, 61.84it/s,
loss=0.1289, acc=95.31%]


 Saved best model with loss: 0.1210

Epoch 90: Loss = 0.1210, Accuracy = 95.31%

Epoch 91/300: 100%|██████████| 44/44 [00:00<00:00, 65.43it/s,
loss=0.1196, acc=95.14%]
Epoch 92/300: 100%|██████████| 44/44 [00:00<00:00, 59.64it/s,
loss=0.2392, acc=95.24%]
Epoch 93/300: 100%|██████████| 44/44 [00:00<00:00, 59.78it/s,
loss=0.0542, acc=94.92%]
Epoch 94/300: 100%|██████████| 44/44 [00:00<00:00, 65.73it/s,
loss=0.0345, acc=95.26%]
```

```
 Saved best model with loss: 0.1204

Epoch 95/300: 100%|██████████| 44/44 [00:00<00:00, 57.62it/s,
loss=0.0837, acc=95.43%]


 Saved best model with loss: 0.1192

Epoch 96/300: 100%|██████████| 44/44 [00:00<00:00, 82.53it/s,
loss=0.0791, acc=94.94%]
Epoch 97/300: 100%|██████████| 44/44 [00:00<00:00, 83.40it/s,
loss=0.1061, acc=95.31%]
Epoch 98/300: 100%|██████████| 44/44 [00:00<00:00, 82.66it/s,
loss=0.0727, acc=95.04%]
Epoch 99/300: 100%|██████████| 44/44 [00:00<00:00, 85.44it/s,
loss=0.0363, acc=95.11%]
Epoch 100/300: 100%|██████████| 44/44 [00:00<00:00, 84.13it/s,
loss=0.1944, acc=95.11%]


Epoch 100: Loss = 0.1220, Accuracy = 95.11%

Epoch 101/300: 100%|██████████| 44/44 [00:00<00:00, 87.25it/s,
loss=0.1230, acc=95.14%]
Epoch 102/300: 100%|██████████| 44/44 [00:00<00:00, 78.55it/s,
loss=0.2048, acc=95.24%]
Epoch 103/300: 100%|██████████| 44/44 [00:00<00:00, 85.83it/s,
loss=0.0592, acc=94.92%]


 Saved best model with loss: 0.1190

Epoch 104/300: 100%|██████████| 44/44 [00:00<00:00, 80.64it/s,
loss=0.1507, acc=95.16%]
Epoch 105/300: 100%|██████████| 44/44 [00:00<00:00, 85.88it/s,
loss=0.1056, acc=95.36%]
Epoch 106/300: 100%|██████████| 44/44 [00:00<00:00, 79.98it/s,
loss=0.2627, acc=95.21%]
Epoch 107/300: 100%|██████████| 44/44 [00:00<00:00, 84.78it/s,
loss=0.1401, acc=95.16%]


 Saved best model with loss: 0.1180

Epoch 108/300: 100%|██████████| 44/44 [00:00<00:00, 79.11it/s,
loss=0.1084, acc=95.38%]
Epoch 109/300: 100%|██████████| 44/44 [00:00<00:00, 81.95it/s,
loss=0.2770, acc=95.16%]
Epoch 110/300: 100%|██████████| 44/44 [00:00<00:00, 83.21it/s,
loss=0.0695, acc=95.24%]
```

```
Epoch 110: Loss = 0.1184, Accuracy = 95.24%

Epoch 111/300: 100%|██████████| 44/44 [00:00<00:00, 81.44it/s,
loss=0.3009, acc=94.77%]
Epoch 112/300: 100%|██████████| 44/44 [00:00<00:00, 83.63it/s,
loss=0.1892, acc=95.16%]
Epoch 113/300: 100%|██████████| 44/44 [00:00<00:00, 84.53it/s,
loss=0.0833, acc=95.04%]
Epoch 114/300: 100%|██████████| 44/44 [00:00<00:00, 61.78it/s,
loss=0.0622, acc=95.24%]


 Saved best model with loss: 0.1166

Epoch 115/300: 100%|██████████| 44/44 [00:00<00:00, 62.81it/s,
loss=0.1177, acc=95.29%]
Epoch 116/300: 100%|██████████| 44/44 [00:00<00:00, 65.97it/s,
loss=0.0530, acc=94.94%]
Epoch 117/300: 100%|██████████| 44/44 [00:00<00:00, 63.58it/s,
loss=0.1807, acc=95.09%]
Epoch 118/300: 100%|██████████| 44/44 [00:00<00:00, 68.00it/s,
loss=0.0803, acc=95.21%]
Epoch 119/300: 100%|██████████| 44/44 [00:00<00:00, 63.90it/s,
loss=0.1526, acc=94.97%]
Epoch 120/300: 100%|██████████| 44/44 [00:00<00:00, 60.47it/s,
loss=0.1032, acc=95.38%]


Epoch 120: Loss = 0.1179, Accuracy = 95.38%

Epoch 121/300: 100%|██████████| 44/44 [00:00<00:00, 79.83it/s,
loss=0.1687, acc=95.02%]
Epoch 122/300: 100%|██████████| 44/44 [00:00<00:00, 83.70it/s,
loss=0.2250, acc=95.24%]
Epoch 123/300: 100%|██████████| 44/44 [00:00<00:00, 82.87it/s,
loss=0.1090, acc=95.21%]
Epoch 124/300: 100%|██████████| 44/44 [00:00<00:00, 78.05it/s,
loss=0.2662, acc=95.26%]


 Saved best model with loss: 0.1155

Epoch 125/300: 100%|██████████| 44/44 [00:00<00:00, 82.39it/s,
loss=0.2237, acc=95.36%]
Epoch 126/300: 100%|██████████| 44/44 [00:00<00:00, 85.61it/s,
loss=0.0623, acc=95.14%]
Epoch 127/300: 100%|██████████| 44/44 [00:00<00:00, 81.59it/s,
loss=0.1591, acc=95.21%]
Epoch 128/300: 100%|██████████| 44/44 [00:00<00:00, 85.02it/s,
loss=0.2991, acc=95.33%]
```

```
Epoch 129/300: 100%|██████████| 44/44 [00:00<00:00, 85.86it/s,
loss=0.1536, acc=95.21%]
Epoch 130/300: 100%|██████████| 44/44 [00:00<00:00, 76.57it/s,
loss=0.0521, acc=95.21%]


 Saved best model with loss: 0.1140

Epoch 130: Loss = 0.1140, Accuracy = 95.21%

Epoch 131/300: 100%|██████████| 44/44 [00:00<00:00, 83.90it/s,
loss=0.0490, acc=95.31%]
Epoch 132/300: 100%|██████████| 44/44 [00:00<00:00, 80.93it/s,
loss=0.0790, acc=95.16%]
Epoch 133/300: 100%|██████████| 44/44 [00:00<00:00, 80.46it/s,
loss=0.0610, acc=95.21%]
Epoch 134/300: 100%|██████████| 44/44 [00:00<00:00, 84.36it/s,
loss=0.1013, acc=95.38%]
Epoch 135/300: 100%|██████████| 44/44 [00:00<00:00, 81.63it/s,
loss=0.1152, acc=95.16%]
Epoch 136/300: 100%|██████████| 44/44 [00:00<00:00, 83.46it/s,
loss=0.0483, acc=95.21%]
Epoch 137/300: 100%|██████████| 44/44 [00:00<00:00, 80.48it/s,
loss=0.1831, acc=95.36%]
Epoch 138/300: 100%|██████████| 44/44 [00:00<00:00, 77.18it/s,
loss=0.2006, acc=95.24%]
Epoch 139/300: 100%|██████████| 44/44 [00:00<00:00, 60.98it/s,
loss=0.0534, acc=95.36%]


 Saved best model with loss: 0.1131

Epoch 140/300: 100%|██████████| 44/44 [00:00<00:00, 64.92it/s,
loss=0.0869, acc=95.26%]


Epoch 140: Loss = 0.1143, Accuracy = 95.26%

Epoch 141/300: 100%|██████████| 44/44 [00:00<00:00, 60.78it/s,
loss=0.0907, acc=95.41%]
Epoch 142/300: 100%|██████████| 44/44 [00:00<00:00, 65.37it/s,
loss=0.1095, acc=95.38%]


 Saved best model with loss: 0.1127

Epoch 143/300: 100%|██████████| 44/44 [00:00<00:00, 67.56it/s,
loss=0.1690, acc=95.26%]
Epoch 144/300: 100%|██████████| 44/44 [00:00<00:00, 58.48it/s,
loss=0.2366, acc=95.24%]
Epoch 145/300: 100%|██████████| 44/44 [00:00<00:00, 80.35it/s,
loss=0.0814, acc=95.31%]
```

```
 Saved best model with loss: 0.1125

Epoch 146/300: 100%|██████████| 44/44 [00:00<00:00, 81.56it/s,
loss=0.0817, acc=95.14%]
Epoch 147/300: 100%|██████████| 44/44 [00:00<00:00, 80.51it/s,
loss=0.1775, acc=95.41%]
Epoch 148/300: 100%|██████████| 44/44 [00:00<00:00, 82.51it/s,
loss=0.1447, acc=95.29%]
Epoch 149/300: 100%|██████████| 44/44 [00:00<00:00, 77.99it/s,
loss=0.0590, acc=95.31%]
Epoch 150/300: 100%|██████████| 44/44 [00:00<00:00, 77.16it/s,
loss=0.1179, acc=95.26%]


 Saved best model with loss: 0.1115

Epoch 150: Loss = 0.1115, Accuracy = 95.26%

Epoch 151/300: 100%|██████████| 44/44 [00:00<00:00, 80.56it/s,
loss=0.0762, acc=95.33%]
Epoch 152/300: 100%|██████████| 44/44 [00:00<00:00, 79.76it/s,
loss=0.0752, acc=95.24%]
Epoch 153/300: 100%|██████████| 44/44 [00:00<00:00, 83.29it/s,
loss=0.1459, acc=95.24%]
Epoch 154/300: 100%|██████████| 44/44 [00:00<00:00, 80.33it/s,
loss=0.2377, acc=95.31%]
Epoch 155/300: 100%|██████████| 44/44 [00:00<00:00, 78.58it/s,
loss=0.1762, acc=95.24%]


 Saved best model with loss: 0.1115

Epoch 156/300: 100%|██████████| 44/44 [00:00<00:00, 80.31it/s,
loss=0.0577, acc=95.29%]
Epoch 157/300: 100%|██████████| 44/44 [00:00<00:00, 83.12it/s,
loss=0.2376, acc=95.14%]
Epoch 158/300: 100%|██████████| 44/44 [00:00<00:00, 81.30it/s,
loss=0.0814, acc=95.29%]
Epoch 159/300: 100%|██████████| 44/44 [00:00<00:00, 76.33it/s,
loss=0.2166, acc=95.21%]
Epoch 160/300: 100%|██████████| 44/44 [00:00<00:00, 83.94it/s,
loss=0.1302, acc=95.19%]


Epoch 160: Loss = 0.1147, Accuracy = 95.19%

Epoch 161/300: 100%|██████████| 44/44 [00:00<00:00, 78.54it/s,
loss=0.1474, acc=95.41%]
Epoch 162/300: 100%|██████████| 44/44 [00:00<00:00, 80.82it/s,
loss=0.1509, acc=95.26%]
```

```
Epoch 163/300: 100%|██████████| 44/44 [00:00<00:00, 58.05it/s,
loss=0.0505, acc=95.43%]
Epoch 164/300: 100%|██████████| 44/44 [00:00<00:00, 65.33it/s,
loss=0.0385, acc=95.33%]
Epoch 165/300: 100%|██████████| 44/44 [00:00<00:00, 59.78it/s,
loss=0.1762, acc=95.36%]
Epoch 166/300: 100%|██████████| 44/44 [00:00<00:00, 61.47it/s,
loss=0.2111, acc=95.46%]
Epoch 167/300: 100%|██████████| 44/44 [00:00<00:00, 70.17it/s,
loss=0.0887, acc=95.38%]


 Saved best model with loss: 0.1109

Epoch 168/300: 100%|██████████| 44/44 [00:00<00:00, 55.82it/s,
loss=0.1937, acc=95.26%]
Epoch 169/300: 100%|██████████| 44/44 [00:00<00:00, 73.89it/s,
loss=0.0514, acc=95.38%]
Epoch 170/300: 100%|██████████| 44/44 [00:00<00:00, 85.04it/s,
loss=0.0839, acc=95.31%]


Epoch 170: Loss = 0.1116, Accuracy = 95.31%

Epoch 171/300: 100%|██████████| 44/44 [00:00<00:00, 82.99it/s,
loss=0.1224, acc=95.38%]
Epoch 172/300: 100%|██████████| 44/44 [00:00<00:00, 83.76it/s,
loss=0.0724, acc=95.43%]
Epoch 173/300: 100%|██████████| 44/44 [00:00<00:00, 77.81it/s,
loss=0.1976, acc=95.33%]
Epoch 174/300: 100%|██████████| 44/44 [00:00<00:00, 83.28it/s,
loss=0.1194, acc=95.31%]
Epoch 175/300: 100%|██████████| 44/44 [00:00<00:00, 81.44it/s,
loss=0.1118, acc=95.29%]
Epoch 176/300: 100%|██████████| 44/44 [00:00<00:00, 83.64it/s,
loss=0.2381, acc=95.19%]
Epoch 177/300: 100%|██████████| 44/44 [00:00<00:00, 83.41it/s,
loss=0.0559, acc=95.26%]
Epoch 178/300: 100%|██████████| 44/44 [00:00<00:00, 83.13it/s,
loss=0.1684, acc=95.46%]
Epoch 179/300: 100%|██████████| 44/44 [00:00<00:00, 83.40it/s,
loss=0.0853, acc=95.53%]
Epoch 180/300: 100%|██████████| 44/44 [00:00<00:00, 84.00it/s,
loss=0.1026, acc=95.48%]


Epoch 180: Loss = 0.1119, Accuracy = 95.48%

Epoch 181/300: 100%|██████████| 44/44 [00:00<00:00, 73.97it/s,
loss=0.0834, acc=95.31%]
Epoch 182/300: 100%|██████████| 44/44 [00:00<00:00, 77.97it/s,
```

```
loss=0.1041, acc=95.36%]
Epoch 183/300: 100%|██████████| 44/44 [00:00<00:00, 80.21it/s,
loss=0.1187, acc=95.46%]
Epoch 184/300: 100%|██████████| 44/44 [00:00<00:00, 81.03it/s,
loss=0.0723, acc=95.36%]
Epoch 185/300: 100%|██████████| 44/44 [00:00<00:00, 76.75it/s,
loss=0.2130, acc=95.38%]
Epoch 186/300: 100%|██████████| 44/44 [00:00<00:00, 80.04it/s,
loss=0.1087, acc=95.43%]


 Saved best model with loss: 0.1109

Epoch 187/300: 100%|██████████| 44/44 [00:00<00:00, 66.08it/s,
loss=0.0643, acc=95.41%]


 Saved best model with loss: 0.1096

Epoch 188/300: 100%|██████████| 44/44 [00:00<00:00, 62.60it/s,
loss=0.0406, acc=95.38%]
Epoch 189/300: 100%|██████████| 44/44 [00:00<00:00, 63.68it/s,
loss=0.1294, acc=95.26%]
Epoch 190/300: 100%|██████████| 44/44 [00:00<00:00, 65.06it/s,
loss=0.2936, acc=95.43%]


Epoch 190: Loss = 0.1129, Accuracy = 95.43%

Epoch 191/300: 100%|██████████| 44/44 [00:00<00:00, 62.67it/s,
loss=0.0864, acc=95.21%]
Epoch 192/300: 100%|██████████| 44/44 [00:00<00:00, 60.05it/s,
loss=0.1215, acc=95.26%]
Epoch 193/300: 100%|██████████| 44/44 [00:00<00:00, 58.54it/s,
loss=0.0357, acc=95.19%]
Epoch 194/300: 100%|██████████| 44/44 [00:00<00:00, 79.75it/s,
loss=0.2505, acc=95.31%]
Epoch 195/300: 100%|██████████| 44/44 [00:00<00:00, 80.09it/s,
loss=0.2331, acc=95.38%]
Epoch 196/300: 100%|██████████| 44/44 [00:00<00:00, 83.33it/s,
loss=0.1508, acc=95.36%]
Epoch 197/300: 100%|██████████| 44/44 [00:00<00:00, 81.62it/s,
loss=0.1123, acc=95.29%]
Epoch 198/300: 100%|██████████| 44/44 [00:00<00:00, 77.40it/s,
loss=0.1767, acc=95.26%]
Epoch 199/300: 100%|██████████| 44/44 [00:00<00:00, 80.09it/s,
loss=0.0673, acc=95.36%]
Epoch 200/300: 100%|██████████| 44/44 [00:00<00:00, 84.40it/s,
loss=0.1412, acc=95.41%]
```

```
Epoch 200: Loss = 0.1110, Accuracy = 95.41%

Epoch 201/300: 100%|██████████| 44/44 [00:00<00:00, 77.64it/s,
loss=0.0911, acc=95.26%]
Epoch 202/300: 100%|██████████| 44/44 [00:00<00:00, 84.47it/s,
loss=0.2243, acc=95.43%]
Epoch 203/300: 100%|██████████| 44/44 [00:00<00:00, 77.86it/s,
loss=0.0939, acc=95.41%]
Epoch 204/300: 100%|██████████| 44/44 [00:00<00:00, 82.59it/s,
loss=0.0740, acc=95.29%]
Epoch 205/300: 100%|██████████| 44/44 [00:00<00:00, 80.69it/s,
loss=0.1657, acc=95.33%]
Epoch 206/300: 100%|██████████| 44/44 [00:00<00:00, 81.60it/s,
loss=0.1623, acc=95.38%]
Epoch 207/300: 100%|██████████| 44/44 [00:00<00:00, 81.96it/s,
loss=0.1131, acc=95.31%]
Epoch 208/300: 100%|██████████| 44/44 [00:00<00:00, 83.76it/s,
loss=0.0593, acc=95.41%]
Epoch 209/300: 100%|██████████| 44/44 [00:00<00:00, 80.66it/s,
loss=0.0755, acc=95.21%]
Epoch 210/300: 100%|██████████| 44/44 [00:00<00:00, 81.41it/s,
loss=0.0795, acc=95.31%]


 Saved best model with loss: 0.1093

Epoch 210: Loss = 0.1093, Accuracy = 95.31%

Epoch 211/300: 100%|██████████| 44/44 [00:00<00:00, 79.95it/s,
loss=0.1851, acc=95.31%]
Epoch 212/300: 100%|██████████| 44/44 [00:00<00:00, 57.24it/s,
loss=0.1527, acc=95.58%]
Epoch 213/300: 100%|██████████| 44/44 [00:00<00:00, 65.41it/s,
loss=0.1408, acc=95.24%]
Epoch 214/300: 100%|██████████| 44/44 [00:00<00:00, 64.00it/s,
loss=0.1193, acc=95.07%]
Epoch 215/300: 100%|██████████| 44/44 [00:00<00:00, 62.63it/s,
loss=0.0926, acc=95.53%]
Epoch 216/300: 100%|██████████| 44/44 [00:00<00:00, 64.44it/s,
loss=0.0620, acc=95.38%]
Epoch 217/300: 100%|██████████| 44/44 [00:00<00:00, 60.90it/s,
loss=0.1035, acc=95.33%]


 Saved best model with loss: 0.1075

Epoch 218/300: 100%|██████████| 44/44 [00:00<00:00, 67.19it/s,
loss=0.2589, acc=95.36%]
Epoch 219/300: 100%|██████████| 44/44 [00:00<00:00, 81.45it/s,
loss=0.0669, acc=95.38%]
```

```
Epoch 220/300: 100%|██████████| 44/44 [00:00<00:00, 80.33it/s,
loss=0.1237, acc=95.38%]


Epoch 220: Loss = 0.1135, Accuracy = 95.38%

Epoch 221/300: 100%|██████████| 44/44 [00:00<00:00, 81.35it/s,
loss=0.1487, acc=95.41%]
Epoch 222/300: 100%|██████████| 44/44 [00:00<00:00, 78.58it/s,
loss=0.1036, acc=95.46%]
Epoch 223/300: 100%|██████████| 44/44 [00:00<00:00, 83.30it/s,
loss=0.0917, acc=95.36%]
Epoch 224/300: 100%|██████████| 44/44 [00:00<00:00, 78.96it/s,
loss=0.0471, acc=95.48%]
Epoch 225/300: 100%|██████████| 44/44 [00:00<00:00, 83.10it/s,
loss=0.1062, acc=95.31%]
Epoch 226/300: 100%|██████████| 44/44 [00:00<00:00, 82.74it/s,
loss=0.0912, acc=95.36%]
Epoch 227/300: 100%|██████████| 44/44 [00:00<00:00, 78.32it/s,
loss=0.0633, acc=95.36%]
Epoch 228/300: 100%|██████████| 44/44 [00:00<00:00, 79.57it/s,
loss=0.1127, acc=95.33%]
Epoch 229/300: 100%|██████████| 44/44 [00:00<00:00, 83.69it/s,
loss=0.0962, acc=95.41%]
Epoch 230/300: 100%|██████████| 44/44 [00:00<00:00, 77.18it/s,
loss=0.1890, acc=95.38%]


Epoch 230: Loss = 0.1144, Accuracy = 95.38%

Epoch 231/300: 100%|██████████| 44/44 [00:00<00:00, 73.79it/s,
loss=0.1476, acc=95.31%]
Epoch 232/300: 100%|██████████| 44/44 [00:00<00:00, 80.41it/s,
loss=0.2224, acc=95.31%]
Epoch 233/300: 100%|██████████| 44/44 [00:00<00:00, 82.18it/s,
loss=0.1385, acc=95.36%]
Epoch 234/300: 100%|██████████| 44/44 [00:00<00:00, 78.99it/s,
loss=0.1348, acc=95.38%]
Epoch 235/300: 100%|██████████| 44/44 [00:00<00:00, 78.47it/s,
loss=0.0125, acc=95.21%]
Epoch 236/300: 100%|██████████| 44/44 [00:00<00:00, 70.12it/s,
loss=0.1409, acc=95.29%]
Epoch 237/300: 100%|██████████| 44/44 [00:00<00:00, 65.28it/s,
loss=0.0954, acc=95.31%]
Epoch 238/300: 100%|██████████| 44/44 [00:00<00:00, 68.65it/s,
loss=0.0551, acc=95.41%]
Epoch 239/300: 100%|██████████| 44/44 [00:00<00:00, 58.51it/s,
loss=0.1336, acc=95.33%]
Epoch 240/300: 100%|██████████| 44/44 [00:00<00:00, 63.49it/s,
loss=0.1248, acc=95.29%]
```

```
Epoch 240: Loss = 0.1113, Accuracy = 95.29%

Epoch 241/300: 100%|██████████| 44/44 [00:00<00:00, 63.11it/s,
loss=0.2112, acc=95.51%]
Epoch 242/300: 100%|██████████| 44/44 [00:00<00:00, 57.94it/s,
loss=0.2579, acc=95.36%]
Epoch 243/300: 100%|██████████| 44/44 [00:00<00:00, 71.82it/s,
loss=0.1502, acc=95.31%]
Epoch 244/300: 100%|██████████| 44/44 [00:00<00:00, 80.74it/s,
loss=0.0595, acc=95.16%]
Epoch 245/300: 100%|██████████| 44/44 [00:00<00:00, 77.62it/s,
loss=0.0881, acc=95.31%]
Epoch 246/300: 100%|██████████| 44/44 [00:00<00:00, 81.75it/s,
loss=0.1076, acc=95.19%]
Epoch 247/300: 100%|██████████| 44/44 [00:00<00:00, 82.31it/s,
loss=0.1170, acc=95.53%]
Epoch 248/300: 100%|██████████| 44/44 [00:00<00:00, 78.29it/s,
loss=0.0927, acc=95.33%]
Epoch 249/300: 100%|██████████| 44/44 [00:00<00:00, 76.95it/s,
loss=0.0900, acc=95.46%]
Epoch 250/300: 100%|██████████| 44/44 [00:00<00:00, 81.72it/s,
loss=0.0692, acc=95.55%]


Epoch 250: Loss = 0.1091, Accuracy = 95.55%

Epoch 251/300: 100%|██████████| 44/44 [00:00<00:00, 80.15it/s,
loss=0.0607, acc=95.26%]
Epoch 252/300: 100%|██████████| 44/44 [00:00<00:00, 85.81it/s,
loss=0.1017, acc=95.46%]
Epoch 253/300: 100%|██████████| 44/44 [00:00<00:00, 81.68it/s,
loss=0.1108, acc=95.29%]
Epoch 254/300: 100%|██████████| 44/44 [00:00<00:00, 80.89it/s,
loss=0.1743, acc=95.26%]
Epoch 255/300: 100%|██████████| 44/44 [00:00<00:00, 82.64it/s,
loss=0.0761, acc=95.43%]
Epoch 256/300: 100%|██████████| 44/44 [00:00<00:00, 84.82it/s,
loss=0.0307, acc=95.24%]
Epoch 257/300: 100%|██████████| 44/44 [00:00<00:00, 81.96it/s,
loss=0.1351, acc=95.29%]
Epoch 258/300: 100%|██████████| 44/44 [00:00<00:00, 85.62it/s,
loss=0.1013, acc=95.36%]
Epoch 259/300: 100%|██████████| 44/44 [00:00<00:00, 79.34it/s,
loss=0.1714, acc=95.33%]
Epoch 260/300: 100%|██████████| 44/44 [00:00<00:00, 84.52it/s,
loss=0.0885, acc=95.26%]


Epoch 260: Loss = 0.1143, Accuracy = 95.26%
```

```
Epoch 261/300: 100%|██████████| 44/44 [00:00<00:00, 71.23it/s,
loss=0.1648, acc=95.31%]
Epoch 262/300: 100%|██████████| 44/44 [00:00<00:00, 63.97it/s,
loss=0.0167, acc=95.51%]
Epoch 263/300: 100%|██████████| 44/44 [00:00<00:00, 68.69it/s,
loss=0.0611, acc=95.24%]
Epoch 264/300: 100%|██████████| 44/44 [00:00<00:00, 59.00it/s,
loss=0.0466, acc=95.43%]
Epoch 265/300: 100%|██████████| 44/44 [00:00<00:00, 66.16it/s,
loss=0.1825, acc=95.46%]
Epoch 266/300: 100%|██████████| 44/44 [00:00<00:00, 69.89it/s,
loss=0.0919, acc=95.38%]
Epoch 267/300: 100%|██████████| 44/44 [00:00<00:00, 55.12it/s,
loss=0.1346, acc=95.29%]
Epoch 268/300: 100%|██████████| 44/44 [00:00<00:00, 76.64it/s,
loss=0.1206, acc=95.51%]
Epoch 269/300: 100%|██████████| 44/44 [00:00<00:00, 83.97it/s,
loss=0.1951, acc=95.41%]
Epoch 270/300: 100%|██████████| 44/44 [00:00<00:00, 80.87it/s,
loss=0.0758, acc=95.21%]


Epoch 270: Loss = 0.1146, Accuracy = 95.21%

Epoch 271/300: 100%|██████████| 44/44 [00:00<00:00, 82.57it/s,
loss=0.0950, acc=95.29%]
Epoch 272/300: 100%|██████████| 44/44 [00:00<00:00, 78.73it/s,
loss=0.0630, acc=95.24%]
Epoch 273/300: 100%|██████████| 44/44 [00:00<00:00, 84.52it/s,
loss=0.1056, acc=95.33%]
Epoch 274/300: 100%|██████████| 44/44 [00:00<00:00, 79.98it/s,
loss=0.1146, acc=95.55%]
Epoch 275/300: 100%|██████████| 44/44 [00:00<00:00, 80.81it/s,
loss=0.1613, acc=95.24%]
Epoch 276/300: 100%|██████████| 44/44 [00:00<00:00, 80.75it/s,
loss=0.0492, acc=95.38%]
Epoch 277/300: 100%|██████████| 44/44 [00:00<00:00, 81.14it/s,
loss=0.0705, acc=95.29%]
Epoch 278/300: 100%|██████████| 44/44 [00:00<00:00, 81.10it/s,
loss=0.0830, acc=95.51%]
Epoch 279/300: 100%|██████████| 44/44 [00:00<00:00, 83.81it/s,
loss=0.2212, acc=95.36%]
Epoch 280/300: 100%|██████████| 44/44 [00:00<00:00, 82.22it/s,
loss=0.1061, acc=95.43%]


Epoch 280: Loss = 0.1146, Accuracy = 95.43%

Epoch 281/300: 100%|██████████| 44/44 [00:00<00:00, 81.45it/s,
loss=0.1317, acc=95.31%]
```

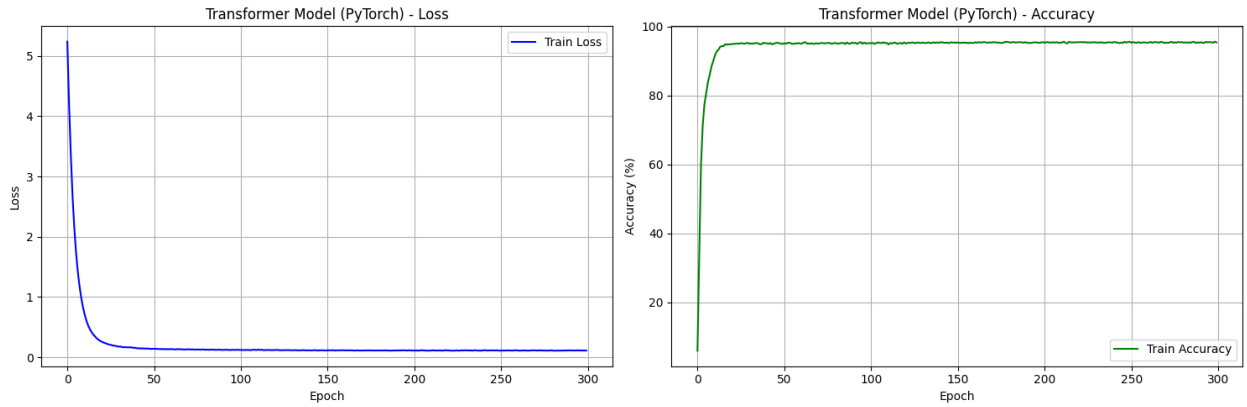
```
Epoch 282/300: 100%|██████████| 44/44 [00:00<00:00, 81.60it/s,
loss=0.0626, acc=95.36%]
Epoch 283/300: 100%|██████████| 44/44 [00:00<00:00, 76.84it/s,
loss=0.0710, acc=95.26%]
Epoch 284/300: 100%|██████████| 44/44 [00:00<00:00, 82.06it/s,
loss=0.0677, acc=95.38%]
Epoch 285/300: 100%|██████████| 44/44 [00:00<00:00, 80.49it/s,
loss=0.0869, acc=95.24%]
Epoch 286/300: 100%|██████████| 44/44 [00:00<00:00, 68.13it/s,
loss=0.0682, acc=95.48%]
Epoch 287/300: 100%|██████████| 44/44 [00:00<00:00, 63.30it/s,
loss=0.1330, acc=95.53%]
Epoch 288/300: 100%|██████████| 44/44 [00:00<00:00, 64.82it/s,
loss=0.1150, acc=95.19%]
Epoch 289/300: 100%|██████████| 44/44 [00:00<00:00, 60.67it/s,
loss=0.0780, acc=95.36%]
Epoch 290/300: 100%|██████████| 44/44 [00:00<00:00, 67.82it/s,
loss=0.1553, acc=95.29%]


Epoch 290: Loss = 0.1136, Accuracy = 95.29%

Epoch 291/300: 100%|██████████| 44/44 [00:00<00:00, 69.70it/s,
loss=0.1313, acc=95.41%]
Epoch 292/300: 100%|██████████| 44/44 [00:00<00:00, 59.68it/s,
loss=0.1177, acc=95.24%]
Epoch 293/300: 100%|██████████| 44/44 [00:00<00:00, 71.42it/s,
loss=0.1244, acc=95.24%]
Epoch 294/300: 100%|██████████| 44/44 [00:00<00:00, 78.51it/s,
loss=0.1209, acc=95.51%]
Epoch 295/300: 100%|██████████| 44/44 [00:00<00:00, 83.19it/s,
loss=0.2367, acc=95.38%]
Epoch 296/300: 100%|██████████| 44/44 [00:00<00:00, 78.02it/s,
loss=0.1082, acc=95.46%]
Epoch 297/300: 100%|██████████| 44/44 [00:00<00:00, 76.78it/s,
loss=0.1300, acc=95.38%]
Epoch 298/300: 100%|██████████| 44/44 [00:00<00:00, 83.16it/s,
loss=0.0904, acc=95.26%]
Epoch 299/300: 100%|██████████| 44/44 [00:00<00:00, 78.15it/s,
loss=0.1117, acc=95.58%]
Epoch 300/300: 100%|██████████| 44/44 [00:00<00:00, 74.97it/s,
loss=0.0867, acc=95.36%]


Epoch 300: Loss = 0.1119, Accuracy = 95.36%
Transformer training completed!
```

```
plot_history(transformer_history, 'Transformer Model (PyTorch)',
f'{WORKING_DIR}/outputs/transformer_training_history.png')
```




```
Plot saved to:
/content/drive/MyDrive/GenAI_Lab_Week10_PyTorch/outputs/transformer_training_history.png

def generate_text_transformer(model, tokenizer, seed_text, length=50, temperature=0.8):
    model.eval()
    generated = seed_text
    current_text = seed_text

    with torch.no_grad():
        for _ in range(length):
            # Encode current text
            encoding = tokenizer.encode(current_text)
            input_ids = encoding.ids[:40]  # Max length
            input_tensor = torch.tensor([input_ids], dtype=torch.long).to(device)

            # Predict
            output = model(input_tensor)  # (1, seq_len, vocab_size)

            # Get last position prediction
            last_output = output[0, len(input_ids)-1, :] / temperature
            probs = F.softmax(last_output, dim=0)

            # Sample
            next_id = torch.multinomial(probs, 1).item()
            next_token = tokenizer.id_to_token(next_id)

            if next_token in ["[PAD]", "[UNK]", "[CLS]", "[SEP]", "[MASK]"]:
                continue

            generated += " " + next_token
            current_text = generated[-200:]  # Keep last 200 chars

    return generated
```

```python
# Load best transformer model
checkpoint =
torch.load(f'{WORKING_DIR}/models/transformer_best_model.pth')
transformer_model.load_state_dict(checkpoint['model_state_dict'])
transformer_model.eval()

print("="*60)
print("TRANSFORMER GENERATED TEXT (PyTorch)")
print("="*60)

transformer_results = {}
for seed in ["artificial intelligence", "machine learning", "deep
learning"]:
    print(f"\nSeed: '{seed}'")
    print("-" * 60)
    for temp in [0.5, 0.7, 0.9]:
        generated = generate_text_transformer(transformer_model,
word_tokenizer, seed, length=30, temperature=temp)
        print(f"\nTemperature {temp}:")
        print(generated)
        transformer_results[f"{seed}_temp{temp}"] = generated

# Save results
with open(f'{WORKING_DIR}/outputs/transformer_generated_text.txt',
'w') as f:
    for key, value in transformer_results.items():
        f.write(f"\n{'='*60}\n{key}\n{'='*60}\n{value}\n")
```

```
============================================================
TRANSFORMER GENERATED TEXT (PyTorch)
============================================================

Seed: 'artificial intelligence'
------------------------------------------------------------

Temperature 0.5:
artificial intelligence systems learn patterns from data ? . from data
. . . ? . . . ! . . . ! . ? . intelligence . . intelligence .
intelligence

Temperature 0.7:
artificial intelligence systems learn patterns from data ! . from data
. dependencies . ##ing ##y . ! . . ##ation learning . . . intelligence
. ? learning data . ##ation

Temperature 0.9:
artificial intelligence systems learn patterns from data ! . data ? .
##ation ##ut improves model accuracy . ! ? . ! . learning . term
dependencies . . intelligence systems learn
```

```
Seed: 'machine learning'
------------------------------------------------------------

Temperature 0.5:
machine learning helps computers learn automatically ! . ##o pro ##duc
. ! ? ? . ? ? ? . ! . . . . . intelligence ! intelligence .
intelligence ?

Temperature 0.7:
machine learning helps computers learn automatically ! . from data .
##o new samples from . ! . ! ? ! . ! . . learning helps computers
learn automatically . learning

Temperature 0.9:
machine learning helps computers learn automatically ! . m ##e . ##v .
? . . ! . ? ! step by step . ! intelligence . . . sentence . ##ation

Seed: 'deep learning'
------------------------------------------------------------

Temperature 0.5:
deep learning models improve sequence learning . . . . human brain
structures . ! . ? . ? . . ! ! . ? ! learning . . intelligence . .

Temperature 0.7:
deep learning models improve sequence learning ! . . and as ##s
##h . ! . . ! . ? . ! . . . ! . ! . . intelligence .

Temperature 0.9:
deep learning models improve sequence learning ! sequence learning .
handle long term dependencies . ? . ! ! . . s ##h ! . intelligence
systems ! . ? . !
```

```python
# Load pre-trained GPT-2 and fine-tune
print("Loading GPT-2 for fine-tuning...")

gpt2_tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
gpt2_tokenizer.pad_token = gpt2_tokenizer.eos_token

# Prepare dataset for GPT-2
class GPT2Dataset(Dataset):
    def __init__(self, texts, tokenizer, max_length=50):
        self.encodings = tokenizer(texts, truncation=True,
max_length=max_length, padding=True, return_tensors='pt')

    def __len__(self):
        return len(self.encodings['input_ids'])

    def __getitem__(self, idx):
        return {key: val[idx] for key, val in self.encodings.items()}
```

```
gpt2_dataset = GPT2Dataset(augmented_sentences, gpt2_tokenizer)
gpt2_dataloader = DataLoader(gpt2_dataset, batch_size=4, shuffle=True)

# Load pre-trained GPT-2
gpt2_model = GPT2LMHeadModel.from_pretrained('gpt2').to(device)

# Freeze first 6 layers (optional - saves memory)
# for param in gpt2_model.transformer.h[:6].parameters():
#     param.requires_grad = False

# Optimizer
gpt2_optimizer = optim.AdamW(gpt2_model.parameters(), lr=5e-5)

# Training loop
def train_gpt2(model, dataloader, optimizer, num_epochs=50):
    model.train()
    best_loss = float('inf')

    for epoch in range(num_epochs):
        total_loss = 0

        progress_bar = tqdm(dataloader, desc=f'GPT-2 Epoch
{epoch+1}/{num_epochs}')

        for batch in progress_bar:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)

            optimizer.zero_grad()

            outputs = model(input_ids, attention_mask=attention_mask,
labels=input_ids)
            loss = outputs.loss

            loss.backward()
            optimizer.step()

            total_loss += loss.item()
            progress_bar.set_postfix({'loss': f'{loss.item():.4f}'})

        avg_loss = total_loss / len(dataloader)
        print(f"\nEpoch {epoch+1}: Average Loss = {avg_loss:.4f}")

        if avg_loss < best_loss:
            best_loss = avg_loss

model.save_pretrained(f'{WORKING_DIR}/models/gpt2_finetuned')

gpt2_tokenizer.save_pretrained(f'{WORKING_DIR}/models/gpt2_finetuned')
            print(f"□ Saved best GPT-2 model")
```

```
# Train (uncomment to run - takes time)
train_gpt2(gpt2_model, gpt2_dataloader, gpt2_optimizer, num_epochs=50)
print("GPT-2 fine-tuning ready (uncomment to run)")
```

```
Loading GPT-2 for fine-tuning...
```

{"model_id":"afbf5bab49394e648465bdd59d02f8bf","version_major":2,"version_minor":0}

```
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                    | Status     |  |
----------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.
GPT-2 Epoch 1/50:   0%|          | 0/20 [00:00<?,
?it/s]`loss_type=None` was set in the config but it is unrecognized.
Using the default loss: `ForCausalLMLoss`.
GPT-2 Epoch 1/50: 100%|██████████| 20/20 [00:02<00:00,  9.92it/s,
loss=1.9578]


Epoch 1: Average Loss = 3.2799
```

{"model_id":"0519e0a28ff1481bbfe0a183f4cad151","version_major":2,"version_minor":0}

```
 Saved best GPT-2 model

GPT-2 Epoch 2/50: 100%|██████████| 20/20 [00:01<00:00, 12.95it/s,
loss=1.1625]


Epoch 2: Average Loss = 1.4456
```

{"model_id":"6f0aac3021344c8fa81fb2cc73b8d38c","version_major":2,"version_minor":0}

```
 Saved best GPT-2 model

GPT-2 Epoch 3/50: 100%|██████████| 20/20 [00:01<00:00, 11.03it/s,
loss=1.0105]


Epoch 3: Average Loss = 0.8377
```

{"model_id":"041ec3a0708b4cba8597b8d22d8e7a96","version_major":2,"version_minor":0}

```
 Saved best GPT-2 model
```

```
GPT-2 Epoch 4/50: 100%|██████████| 20/20 [00:01<00:00, 12.79it/s,
loss=0.3626]


Epoch 4: Average Loss = 0.6126

{"model_id":"b259861f323a4e2eba8875eed29bdfaa","version_major":2,"vers
ion_minor":0}

 Saved best GPT-2 model

GPT-2 Epoch 5/50: 100%|██████████| 20/20 [00:01<00:00, 12.88it/s,
loss=0.7132]


Epoch 5: Average Loss = 0.5876

{"model_id":"3c9bf5f3814349bdaa45ccd8834db949","version_major":2,"vers
ion_minor":0}

 Saved best GPT-2 model

GPT-2 Epoch 6/50: 100%|██████████| 20/20 [00:01<00:00, 11.14it/s,
loss=0.5023]


Epoch 6: Average Loss = 0.5409

{"model_id":"e616d4eed57d43209608129a0858e41a","version_major":2,"vers
ion_minor":0}

 Saved best GPT-2 model

GPT-2 Epoch 7/50: 100%|██████████| 20/20 [00:01<00:00, 13.00it/s,
loss=0.5037]


Epoch 7: Average Loss = 0.5468

GPT-2 Epoch 8/50: 100%|██████████| 20/20 [00:01<00:00, 13.01it/s,
loss=0.1421]


Epoch 8: Average Loss = 0.4352

{"model_id":"8f3565251e2449a5b17daf09d77b4e61","version_major":2,"vers
ion_minor":0}

 Saved best GPT-2 model

GPT-2 Epoch 9/50: 100%|██████████| 20/20 [00:01<00:00, 10.96it/s,
loss=0.2877]
```

```
Epoch 9: Average Loss = 0.4282

{"model_id":"1de654f4d226452384bec02e5edde445","version_major":2,"version_minor":0}

 Saved best GPT-2 model

GPT-2 Epoch 10/50: 100%|██████████| 20/20 [00:01<00:00, 12.92it/s, loss=0.5279]


Epoch 10: Average Loss = 0.3893

{"model_id":"a3c088e71d1d44608a84642fbebae447","version_major":2,"version_minor":0}

 Saved best GPT-2 model

GPT-2 Epoch 11/50: 100%|██████████| 20/20 [00:01<00:00, 12.06it/s, loss=0.6703]


Epoch 11: Average Loss = 0.5138

GPT-2 Epoch 12/50: 100%|██████████| 20/20 [00:01<00:00, 11.13it/s, loss=0.3259]


Epoch 12: Average Loss = 0.4274

GPT-2 Epoch 13/50: 100%|██████████| 20/20 [00:01<00:00, 11.51it/s, loss=0.3564]


Epoch 13: Average Loss = 0.4450

GPT-2 Epoch 14/50: 100%|██████████| 20/20 [00:01<00:00, 12.99it/s, loss=0.8128]


Epoch 14: Average Loss = 0.4139

GPT-2 Epoch 15/50: 100%|██████████| 20/20 [00:01<00:00, 12.25it/s, loss=0.4117]


Epoch 15: Average Loss = 0.3842

{"model_id":"55aa230521584f8c93a2ebcae4c00676","version_major":2,"version_minor":0}

 Saved best GPT-2 model
```

```
GPT-2 Epoch 16/50: 100%|██████████| 20/20 [00:01<00:00, 12.78it/s, loss=0.8086]


Epoch 16: Average Loss = 0.4541

GPT-2 Epoch 17/50: 100%|██████████| 20/20 [00:01<00:00, 12.94it/s, loss=0.2644]


Epoch 17: Average Loss = 0.3403

{"model_id":"e0d0dc33fd134d86acef66cc54f088bf","version_major":2,"version_minor":0}

 Saved best GPT-2 model

GPT-2 Epoch 18/50: 100%|██████████| 20/20 [00:01<00:00, 11.10it/s, loss=0.4301]


Epoch 18: Average Loss = 0.4193

GPT-2 Epoch 19/50: 100%|██████████| 20/20 [00:01<00:00, 11.96it/s, loss=0.4355]


Epoch 19: Average Loss = 0.3758

GPT-2 Epoch 20/50: 100%|██████████| 20/20 [00:01<00:00, 12.91it/s, loss=0.3076]


Epoch 20: Average Loss = 0.3616

GPT-2 Epoch 21/50: 100%|██████████| 20/20 [00:01<00:00, 12.94it/s, loss=0.6845]


Epoch 21: Average Loss = 0.3443

GPT-2 Epoch 22/50: 100%|██████████| 20/20 [00:01<00:00, 11.54it/s, loss=0.2403]


Epoch 22: Average Loss = 0.3524

GPT-2 Epoch 23/50: 100%|██████████| 20/20 [00:01<00:00, 10.20it/s, loss=0.6191]


Epoch 23: Average Loss = 0.3841
```

```
GPT-2 Epoch 24/50: 100%|██████████| 20/20 [00:02<00:00,  9.72it/s, loss=0.4804]


Epoch 24: Average Loss = 0.4191

GPT-2 Epoch 25/50: 100%|██████████| 20/20 [00:01<00:00, 11.28it/s, loss=0.1716]


Epoch 25: Average Loss = 0.2936

{"model_id":"60bfdb49501f4574850913bfe057efa8","version_major":2,"version_minor":0}

 Saved best GPT-2 model

GPT-2 Epoch 26/50: 100%|██████████| 20/20 [00:01<00:00, 12.86it/s, loss=0.1906]


Epoch 26: Average Loss = 0.3665

GPT-2 Epoch 27/50: 100%|██████████| 20/20 [00:01<00:00, 12.42it/s, loss=0.3938]


Epoch 27: Average Loss = 0.3501

GPT-2 Epoch 28/50: 100%|██████████| 20/20 [00:01<00:00, 11.19it/s, loss=0.2300]


Epoch 28: Average Loss = 0.3304

GPT-2 Epoch 29/50: 100%|██████████| 20/20 [00:01<00:00, 10.90it/s, loss=0.1590]


Epoch 29: Average Loss = 0.2581

{"model_id":"4ed1b61e9b7e4538b4ecfb0065c0392c","version_major":2,"version_minor":0}

 Saved best GPT-2 model

GPT-2 Epoch 30/50: 100%|██████████| 20/20 [00:01<00:00, 13.03it/s, loss=0.1405]


Epoch 30: Average Loss = 0.2991

GPT-2 Epoch 31/50: 100%|██████████| 20/20 [00:01<00:00, 12.97it/s, loss=0.1891]
```

```
Epoch 31: Average Loss = 0.3037

GPT-2 Epoch 32/50: 100%|██████████| 20/20 [00:01<00:00, 12.89it/s,
loss=0.2655]


Epoch 32: Average Loss = 0.3238

GPT-2 Epoch 33/50: 100%|██████████| 20/20 [00:01<00:00, 12.20it/s,
loss=0.1999]


Epoch 33: Average Loss = 0.3671

GPT-2 Epoch 34/50: 100%|██████████| 20/20 [00:02<00:00,  8.92it/s,
loss=0.2914]


Epoch 34: Average Loss = 0.2954

GPT-2 Epoch 35/50: 100%|██████████| 20/20 [00:02<00:00,  9.31it/s,
loss=0.3582]


Epoch 35: Average Loss = 0.2733

GPT-2 Epoch 36/50: 100%|██████████| 20/20 [00:01<00:00, 11.44it/s,
loss=0.8998]


Epoch 36: Average Loss = 0.3775

GPT-2 Epoch 37/50: 100%|██████████| 20/20 [00:01<00:00, 11.64it/s,
loss=0.1895]


Epoch 37: Average Loss = 0.3647

GPT-2 Epoch 38/50: 100%|██████████| 20/20 [00:01<00:00, 11.24it/s,
loss=0.4153]


Epoch 38: Average Loss = 0.3330

GPT-2 Epoch 39/50: 100%|██████████| 20/20 [00:01<00:00, 13.05it/s,
loss=0.3643]


Epoch 39: Average Loss = 0.3229

GPT-2 Epoch 40/50: 100%|██████████| 20/20 [00:01<00:00, 12.14it/s,
loss=0.2295]
```

```
Epoch 40: Average Loss = 0.3085

GPT-2 Epoch 41/50: 100%|██████████| 20/20 [00:02<00:00,  9.57it/s,
loss=0.4373]


Epoch 41: Average Loss = 0.3158

GPT-2 Epoch 42/50: 100%|██████████| 20/20 [00:02<00:00,  7.35it/s,
loss=0.6739]


Epoch 42: Average Loss = 0.3586

GPT-2 Epoch 43/50: 100%|██████████| 20/20 [00:02<00:00,  9.81it/s,
loss=0.5638]


Epoch 43: Average Loss = 0.2918

GPT-2 Epoch 44/50: 100%|██████████| 20/20 [00:01<00:00, 13.09it/s,
loss=0.3193]


Epoch 44: Average Loss = 0.2931

GPT-2 Epoch 45/50: 100%|██████████| 20/20 [00:01<00:00, 13.31it/s,
loss=0.1126]


Epoch 45: Average Loss = 0.2883

GPT-2 Epoch 46/50: 100%|██████████| 20/20 [00:01<00:00, 13.13it/s,
loss=0.2918]


Epoch 46: Average Loss = 0.3046

GPT-2 Epoch 47/50: 100%|██████████| 20/20 [00:01<00:00, 13.06it/s,
loss=0.1384]


Epoch 47: Average Loss = 0.2080



{"model_id":"d126143c65e04ab3a0f9e98e4abd8610","version_major":2,"vers
ion_minor":0}

 Saved best GPT-2 model

GPT-2 Epoch 48/50: 100%|██████████| 20/20 [00:01<00:00, 11.27it/s,
loss=0.3375]
```

```
Epoch 48: Average Loss = 0.2714

GPT-2 Epoch 49/50: 100%|██████████| 20/20 [00:01<00:00, 11.15it/s,
loss=0.3754]


Epoch 49: Average Loss = 0.2953

GPT-2 Epoch 50/50: 100%|██████████| 20/20 [00:01<00:00, 12.51it/s,
loss=0.2193]


Epoch 50: Average Loss = 0.2586
GPT-2 fine-tuning ready (uncomment to run)
```

```python
def generate_with_gpt2(model, tokenizer, prompt, max_length=100,
temperature=0.8):
    model.eval()
    input_ids = tokenizer.encode(prompt,
return_tensors='pt').to(device)

    with torch.no_grad():
        output = model.generate(
            input_ids,
            max_length=max_length,
            temperature=temperature,
            top_k=50,
            top_p=0.95,
            repetition_penalty=1.2,
            do_sample=True,
            num_return_sequences=1,
            pad_token_id=tokenizer.eos_token_id
        )

    return tokenizer.decode(output[0], skip_special_tokens=True)

# Load fine-tuned model if exists
try:
    gpt2_finetuned =
GPT2LMHeadModel.from_pretrained(f'{WORKING_DIR}/models/gpt2_finetuned'
).to(device)
    gpt2_tokenizer_ft =
GPT2Tokenizer.from_pretrained(f'{WORKING_DIR}/models/gpt2_finetuned')

    print("="*60)
    print("FINE-TUNED GPT-2 GENERATED TEXT")
    print("="*60)
```

```
    gpt2_results = {}
    for seed in ["artificial intelligence", "machine learning",
"neural networks"]:
        print(f"\nPrompt: '{seed}'")
        generated = generate_with_gpt2(gpt2_finetuned,
gpt2_tokenizer_ft, seed, max_length=80)
        print(generated)

        gpt2_results[seed] = generated

    with open(f'{WORKING_DIR}/outputs/gpt2_generated_text.txt', 'w')
as f:
        for key, value in gpt2_results.items():
            f.write(f"\n{'='*60}\n{key}\n{'='*60}\n{value}\n")
except:
    print("No fine-tuned GPT-2 model found. Run Cell 18 first.")
```

{"model_id":"b1c4725a016d4502a27b68805a5c8275","version_major":2,"version_minor":0}

```
============================================================
FINE-TUNED GPT-2 GENERATED TEXT
============================================================

Prompt: 'artificial intelligence'
artificial intelligence systems learn patterns from data!

Prompt: 'machine learning'
machine learning helps computers learn automatically!

Prompt: 'neural networks'
neural networks simulate human brain structures.
```

```
def calculate_perplexity(model, dataloader, criterion,
model_type='lstm'):
    model.eval()
    total_loss = 0
    total_tokens = 0

    with torch.no_grad():
        for inputs, targets in dataloader:
            inputs, targets = inputs.to(device), targets.to(device)

            if model_type == 'lstm':
                outputs = model(inputs)
                loss = criterion(outputs, targets)
                total_loss += loss.item() * inputs.size(0)
                total_tokens += inputs.size(0)
            else:  # transformer
                outputs = model(inputs)
                outputs_flat = outputs.view(-1, outputs.size(-1))
```

```
                targets_flat = targets.view(-1)
                mask = targets_flat != 0
                loss = criterion(outputs_flat, targets_flat)
                total_loss += loss.item() * mask.sum().item()
                total_tokens += mask.sum().item()

    avg_loss = total_loss / total_tokens
    perplexity = math.exp(avg_loss)
    return perplexity

print("="*60)
print("MODEL EVALUATION & COMPARISON (PyTorch)")
print("="*60)

# LSTM Evaluation
lstm_perplexity = calculate_perplexity(lstm_model, char_dataloader,
criterion, 'lstm')
print(f"\nLSTM Model:")
print(f"  - Perplexity: {lstm_perplexity:.2f}")

# Transformer Evaluation
trans_perplexity = calculate_perplexity(transformer_model,
word_dataloader, criterion_trans, 'transformer')
print(f"\nTransformer Model:")
print(f"  - Perplexity: {trans_perplexity:.2f}")

# Save evaluation results
evaluation_results = {
    'LSTM': {
        'perplexity': float(lstm_perplexity)
    },
    'Transformer': {
        'perplexity': float(trans_perplexity)
    }
}

with open(f'{WORKING_DIR}/evaluation_results.json', 'w') as f:
    json.dump(evaluation_results, f, indent=2)

print(f"\nResults saved to: {WORKING_DIR}/evaluation_results.json")
```

```
============================================================
MODEL EVALUATION & COMPARISON (PyTorch)
============================================================

LSTM Model:
  - Perplexity: 1.08

Transformer Model:
  - Perplexity: 1.11
```

```
Results saved to:
/content/drive/MyDrive/GenAI_Lab_Week10_PyTorch/evaluation_results.jso
n

# Generate comprehensive report
report = f"""
{'='*80}
GENERATIVE AI LAB 10 - FINAL REPORT (PyTorch)
Generated on: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}
{'='*80}

1. DATASET INFORMATION
{'-'*40}
- Original sentences: {len(sentences)}
- Augmented sentences: {len(augmented_sentences)}
- Character vocabulary size: {char_tokenizer.vocab_size}
- Word vocabulary size: {word_tokenizer.get_vocab_size()}

2. MODEL ARCHITECTURES
{'-'*40}
LSTM Model:
- Embedding dimension: 128
- Hidden dimension: 256
- LSTM layers: 3
- Total parameters: {sum(p.numel() for p in
lstm_model.parameters()):,}

Transformer Model:
- Embedding dimension: 128
- Attention heads: 4
- Feed-forward dimension: 512
- Number of layers: 3
- Total parameters: {sum(p.numel() for p in
transformer_model.parameters()):,}

3. TRAINING PERFORMANCE
{'-'*40}
LSTM:
- Final training loss: {lstm_history['train_loss'][-1]:.4f}
- Best training accuracy: {max(lstm_history['train_acc']):.2f}%

Transformer:
- Final training loss: {transformer_history['train_loss'][-1]:.4f}
- Best training accuracy: {max(transformer_history['train_acc']):.2f}%

4. EVALUATION METRICS
{'-'*40}
LSTM:
- Perplexity: {lstm_perplexity:.2f}
```

```
Transformer:
- Perplexity: {trans_perplexity:.2f}

5. GENERATED SAMPLES
{'-'*40}
See outputs folder for generated text samples with different
temperatures.

6. FILES SAVED
{'-'*40}
All models, tokenizers, histories, and outputs saved to:
{WORKING_DIR}/

{'='*80}
"""

print(report)

# Save report
with open(f'{WORKING_DIR}/FINAL_REPORT.txt', 'w') as f:
    f.write(report)

print(f"\n□ COMPLETE! All files saved to: {WORKING_DIR}")
print("□ Directory structure:")
for root, dirs, files in os.walk(WORKING_DIR):
    level = root.replace(WORKING_DIR, '').count(os.sep)
    indent = ' ' * 2 * level
    print(f'{indent}{os.path.basename(root)}/')
    subindent = ' ' * 2 * (level + 1)
    for file in files:
        print(f'{subindent}{file}')
```

```
================================================================================
GENERATIVE AI LAB 10 - FINAL REPORT (PyTorch)
Generated on: 2026-03-24 09:10:25
================================================================================

1. DATASET INFORMATION
----------------------------------------
- Original sentences: 16
- Augmented sentences: 79
- Character vocabulary size: 32
- Word vocabulary size: 300

2. MODEL ARCHITECTURES
----------------------------------------
```

```
LSTM Model:
- Embedding dimension: 128
- Hidden dimension: 256
- LSTM layers: 3
- Total parameters: 1,555,360

Transformer Model:
- Embedding dimension: 128
- Attention heads: 4
- Feed-forward dimension: 512
- Number of layers: 3
- Total parameters: 671,916

3. TRAINING PERFORMANCE
----------------------------------------
LSTM:
- Final training loss: 0.0871
- Best training accuracy: 96.82%

Transformer:
- Final training loss: 0.1119
- Best training accuracy: 95.58%

4. EVALUATION METRICS
----------------------------------------
LSTM:
- Perplexity: 1.08

Transformer:
- Perplexity: 1.11

5. GENERATED SAMPLES
----------------------------------------
See outputs folder for generated text samples with different
temperatures.

6. FILES SAVED
----------------------------------------
All models, tokenizers, histories, and outputs saved to:
/content/drive/MyDrive/GenAI_Lab_Week10_PyTorch/

======================================================================
==========


 COMPLETE! All files saved to:
/content/drive/MyDrive/GenAI_Lab_Week10_PyTorch
 Directory structure:
GenAI_Lab_Week10_PyTorch/
  augmented_data.txt
```

```
  char_tokenizer.pkl
  lstm_history.pkl
  word_tokenizer.json
  transformer_history.pkl
  evaluation_results.json
  FINAL_REPORT.txt
  models/
    lstm_best_model.pth
    lstm_final_model.pth
    transformer_best_model.pth
    transformer_final_model.pth
    gpt2_finetuned/
      config.json
      generation_config.json
      model.safetensors
      tokenizer_config.json
      tokenizer.json
  outputs/
    lstm_training_history.png
    lstm_generated_text.txt
    transformer_training_history.png
    transformer_generated_text.txt
    gpt2_generated_text.txt
```